

Laporan Tugas Kecil 3

IF2211 Strategi Algoritma



Dipersiapkan oleh:

13524005 - Geraldo Artemius

13524055 - Junior Natra Situmorang

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132**

2026

DAFTAR ISI

Laporan Tugas Kecil 3.....	1
DAFTAR ISI.....	2
DAFTAR TABEL.....	3
BAB I.....	4
1.1 Pseudocode Program.....	4
1.2 Algoritma UCS.....	7
1.3 Algoritma GBFS.....	7
1.4 Algoritma A*.....	8
1.5 Algoritma BFS (Bonus).....	8
1.6 Algoritma DFS (Bonus).....	9
BAB II.....	10
2.1 Analisis Algoritma PathFinding.....	10
2.1.1 Analisis Algoritma UCS.....	10
2.1.2 Analisis Algoritma GBFS.....	10
2.1.3 Analisis Algoritma A*.....	11
2.1.4 Analisis Algoritma BFS.....	12
2.1.5 Analisis Algoritma DFS.....	12
2.1.6 Perbandingan Algoritma PathFinding.....	12
BAB III.....	14
3.1 Struktur Folder.....	14
3.2 Source Program.....	14
3.2.1 Main.java.....	14
3.2.2 Algorithm.java.....	18
3.2.3 State.java.....	28
3.2.4 Map.java.....	29
BAB IV.....	33
4.1 Test Case 1.....	33
4.2 Test Case 2.....	47
4.3 Test Case 3.....	60
4.4 Test Case 4.....	69
4.5 Test Case 5.....	70
4.6 Test Case 6.....	80
BAB V.....	96
5.1 Analisis hasil percobaan Algoritma PathFinding.....	96
LAMPIRAN.....	99

DAFTAR TABEL

Tabel 2.6.1. Perbandingan Algoritma PathFinding (UCS, GBFS, A*, BFS, DFS).....	12
Tabel 3.1. Struktur Folder Tree.....	14
Tabel 5.1. Hasil Test PathFinding.....	64

BAB I

ALGORITMA *PATHFINDING*

1.1 Pseudocode Program

Berikut adalah pseudocode dari algoritma UCS, GBFS dan A*.

```
FUNCTION solveBestFirst(algorithm) → State
    queue ← NEW PriorityQueue()
    visited[rowCount][colCount][11] ← ARRAY of boolean (ALL FALSE)
    totalIteration ← 0

    OPEN file iterationFilePath FOR writing

    startState ← NEW State(initialX, initialY, G=0, F=0, steps="",
                           curDigit=0)
    queue.PUSH(startState)

    WHILE queue IS NOT EMPTY DO
        cur ← queue.POLL()
        totalIteration ← totalIteration + 1

        TULIS "=== Iteration " + totalIteration + " ===" KE file
        TULIS visualisasi board berdasarkan cur.steps KE file

        IF board[cur.X][cur.Y] = 'O' AND cur.curDigit > maxDigit THEN
            TULIS "Found Solution" KE file
            RETURN cur
        END IF

        IF visited[cur.X][cur.Y][cur.curDigit] THEN
            CONTINUE
        END IF

        visited[cur.X][cur.Y][cur.curDigit] ← TRUE

        FOR i ← 0 TO 3 DO
            nextState ← slide(cur, dx[i], dy[i], dir[i], algorithm,
                             file)

            IF nextState ≠ NULL AND NOT visited[nextState.X]
               [nextState.Y][nextState.curDigit] THEN
                queue.PUSH(nextState)
            END IF
        END FOR
    END WHILE

    CLOSE file
    RETURN NULL
END FUNCTION

FUNCTION slide(cur, dirX, dirY, dirChar, algorithm, file) → State
```

```

x ← cur.X
y ← cur.Y
costStep ← 0
curTarget ← cur.curDigit
moved ← FALSE
prevTarget ← curTarget

WHILE TRUE DO
    newX ← x + dirX
    newY ← y + dirY

    IF newX, newY di luar batas board THEN
        TULIS dirChar + " GAME OVER (Keluar batas)" KE file
        RETURN NULL
    END IF

    tile ← board[newX][newY]

    IF tile = 'X' THEN BREAK

    IF tile = 'L' THEN
        TULIS dirChar + " GAME OVER (Lava)" KE file
        RETURN NULL
    END IF

    x ← newX
    y ← newY
    costStep ← costStep + boardCosts[x][y]
    moved ← TRUE

    IF tile ADALAH Digit THEN
        digitValue ← ToInt(tile)
        IF digitValue = curTarget THEN
            curTarget ← curTarget + 1
        ELSE IF digitValue > curTarget THEN
            TULIS dirChar + " GAME OVER (Urutan salah)" KE file
            RETURN NULL
        END IF
    END IF
END WHILE

IF NOT moved THEN RETURN NULL

newG ← cur.G + costStep

IF algorithm = "BFS" OR algorithm = "DFS" THEN
    newF ← newG
ELSE
    newH ← heuristicCost(x, y, curTarget, algorithm)
    IF algorithm = "GBFS" THEN
        newF ← newH
    ELSE
        newF ← newG + newH
    END IF
END IF

```

```

        END IF
    END IF

    isTargetFulfilled ← (curTarget > prevTarget)
    RETURN NEW State(x, y, newG, newF, cur.steps + dirChar,
curTarget, isTargetFulfilled)
END FUNCTION

FUNCTION heuristicCost(x, y, targetDigit, algorithm) → integer
    IF algorithm = "UCS" THEN RETURN 0

    targetX, targetY ← Koordinat Finish 'O'

    IF targetDigit ≤ maxDigit THEN
        CARI lokasi (i, j) pada board di mana board[i][j] =
targetDigit
        targetX ← i
        targetY ← j
    END IF

    manhattan ← |x - targetX| + |y - targetY|

    CASE heuristicType OF
        "H2": RETURN manhattan * minCost
        "H3":
            distToTarget ← manhattan
            distFinish ← |targetX - finishX| + |targetY - finishY|
            RETURN (distToTarget + distFinish) * minCost
        DEFAULT:
            RETURN manhattan * minCost
    END CASE
END FUNCTION

```

Pada tugas ini algoritma pathfinding UCS, GBFS dan A* menjadi algoritma utama dalam permasalahan **Ice Sliding Puzzle Solver**. Setiap algoritma memiliki alur awal serupa, mula - mula user akan melakukan input berupa nama file.txt yang akan digunakan sebagai input papan. Format file dapat dilihat sebagai berikut.

test1.txt

```

7 7
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3   5   2   8   1   999
999 7   4   999 6   9   999
999 2   8   3   5   4   999

```

999	6	1	7	2	999	999
999	9	3	4	999	8	999
999	999	999	999	999	999	999

Setelah menerima file input user akan melakukan input algoritma yang akan digunakan, seperti UCS, GBFS, A*, BFS dan DFS. setelah user melakukan input salah satu dari kelima algoritma tersebut user diminta melakukan input heuristik (H1-H3) apabila memilih GBFS ataupun A* hal ini dikarenakan kedua algoritma ini memerlukan Heuristik untuk digunakan dalam penyelesaiannya. Lalu akan masuk ke salah satu dari kelima algoritma tersebut.

1.2 Algoritma UCS

Apabila user memilih menggunakan algoritma UCS, maka proses akan dijalankan melalui fungsi *solveBestFirst*. Fungsi ini merupakan inti dari penyelesaian masalah dengan pendekatan *Best-First Search*. Pada tahap awal, fungsi akan membuat sebuah *Priority Queue* yang digunakan untuk menyimpan state, kemudian state awal dimasukkan dengan nilai $F = 0$ (karena belum ada cost yang dikeluarkan).

Selanjutnya, algoritma akan melakukan pengulangan selama *queue* tidak kosong atau hingga solusi ditemukan. Pada setiap iterasi, algoritma akan mengambil node dengan nilai F terkecil dari *queue*, yang dalam kasus UCS berarti node dengan cost terkecil dari titik awal. Setelah itu, dilakukan pengecekan apakah node tersebut telah mencapai goal. Jika sudah, maka algoritma berhenti dan solusi dikembalikan. Namun, jika belum mencapai goal, maka node tersebut akan:

- Ditandai sebagai visited untuk menghindari pengulangan eksplorasi
- Menghasilkan semua kemungkinan neighbor (pergerakan U, D, L, R)

Untuk setiap neighbor yang dihasilkan, akan dihitung:

- $G \text{ baru} = G \text{ sebelumnya} + \text{cost pergerakan}$
- $F = G$ (karena UCS tidak menggunakan heuristic)

Neighbor tersebut kemudian dimasukkan ke dalam *Priority Queue* selama belum pernah dikunjungi sebelumnya. Proses ini akan terus berulang hingga solusi ditemukan atau semua kemungkinan telah dieksplorasi.

1.3 Algoritma GBFS

Apabila user memilih menggunakan algoritma GBFS, maka proses akan dijalankan melalui fungsi *solveBestFirst* dengan parameter `algorithm="GBFS"`. Fungsi ini merupakan varian dari Best-First Search yang mengedepankan greedy approach berbasis heuristic.

Pada tahap awal, fungsi akan membuat sebuah *Priority Queue* untuk menyimpan state, kemudian state awal dimasukkan dengan nilai $F = 0$. Selanjutnya, algoritma akan melakukan pengulangan selama *queue* tidak kosong atau hingga solusi ditemukan.

Pada setiap iterasi, algoritma akan mengambil node dengan nilai F terkecil dari *queue*. Dalam hal ini, F merepresentasikan nilai heuristic saja (H), tanpa mempertimbangkan cost sebenarnya dari titik awal. Setelah itu, dilakukan pengecekan apakah node tersebut telah mencapai goal (pada posisi 'O' dan semua digit sudah dikumpulkan). Jika belum mencapai goal, maka node tersebut akan:

- Ditandai sebagai visited untuk menghindari pengulangan eksplorasi
- Menghasilkan semua kemungkinan neighbor (pergerakan U, D, L, R)

Untuk setiap neighbor yang dihasilkan, akan dihitung:

- $G \text{ baru} = G \text{ sebelumnya} + \text{cost pergerakan}$ (disimpan namun tidak digunakan dalam prioritas)
- H = heuristic cost menuju digit target berikutnya (berdasarkan heuristicType: H1, H2, atau H3)
- $F = H$ saja (mengabaikan G) - inilah perbedaan utama GBFS dengan A^*

Neighbor kemudian dimasukkan ke Priority Queue selama belum pernah dikunjungi sebelumnya. Proses berulang hingga solusi ditemukan atau semua kemungkinan dieksplorasi.

1.4 Algoritma A^*

Apabila user memilih menggunakan algoritma A^* , maka proses akan dijalankan melalui fungsi solveBestFirst dengan parameter `algorithm="A"`. Fungsi ini merupakan varian Best-First Search yang menggabungkan keuntungan UCS dan GBFS dengan mempertimbangkan kedua faktor: cost sebenarnya dan estimasi heuristic.

Pada tahap awal, fungsi membuat Priority Queue dan memasukkan state awal dengan nilai $F = 0$. Algoritma melakukan pengulangan selama queue tidak kosong atau hingga solusi ditemukan.

Pada setiap iterasi, algoritma mengambil node dengan nilai F terkecil dari queue. Nilai F ini merepresentasikan kombinasi dari cost sebenarnya (G) dan estimasi cost menuju goal (H). Setelah itu, dilakukan pengecekan apakah node sudah mencapai goal. Jika belum mencapai goal, maka node akan:

- Ditandai sebagai visited untuk menghindari pengulangan eksplorasi
- Menghasilkan semua kemungkinan neighbor (pergerakan U, D, L, R)

Untuk setiap neighbor yang dihasilkan, akan dihitung:

- $G \text{ baru} = G \text{ sebelumnya} + \text{cost pergerakan}$ (cost sebenarnya dari start)
- H = heuristic cost menuju digit target berikutnya (berdasarkan heuristicType: H1, H2, atau H3)
- $F = G + H$ (kombinasi kedua faktor) - inilah formula inti A^*

Neighbor dimasukkan ke Priority Queue selama belum pernah dikunjungi sebelumnya. Proses berulang hingga solusi ditemukan.

1.5 Algoritma BFS (Bonus)

Apabila user memilih menggunakan algoritma BFS, maka proses akan dijalankan melalui fungsi solveBFS() yang berbeda dari fungsi-fungsi sebelumnya. BFS menggunakan struktur Queue (FIFO - First In First Out) bukan Priority Queue.

Pada tahap awal, fungsi membuat sebuah Queue biasa dan memasukkan state awal ke dalamnya. Algoritma melakukan pengulangan selama queue tidak kosong atau hingga solusi ditemukan. Pada setiap iterasi, algoritma mengambil node yang pertama kali dimasukkan (FIFO), bukan yang memiliki F terkecil. Setelah itu, dilakukan pengecekan apakah node tersebut telah mencapai goal. Jika belum mencapai goal, maka node akan:

- Ditandai sebagai visited untuk menghindari pengulangan eksplorasi
- Menghasilkan semua kemungkinan neighbor (pergerakan U, D, L, R)

Untuk setiap neighbor yang dihasilkan akan dihitung:

- $G \text{ baru} = G \text{ sebelumnya} + \text{cost pergerakan}$

- $F = G$ (sama seperti UCS, namun urutan eksplorasi berbeda)
- Neighbor dimasukkan ke queue selama belum pernah dikunjungi

Proses berulang hingga solusi ditemukan atau semua kemungkinan dieksplorasi.

1.6 Algoritma DFS (Bonus)

Apabila user memilih menggunakan algoritma DFS, maka proses akan dijalankan melalui fungsi solveDFS(). DFS menggunakan struktur Deque sebagai Stack (LIFO - Last In First Out).

Pada tahap awal, fungsi membuat sebuah Deque (Double Ended Queue) dan memasukkan state awal ke dalamnya menggunakan push(). Algoritma melakukan pengulangan selama stack tidak kosong atau hingga solusi ditemukan. Pada setiap iterasi, algoritma mengambil node yang terakhir kali dimasukkan (LIFO) menggunakan pop(), bukan yang pertama kali dimasukkan. Setelah itu, dilakukan pengecekan apakah node tersebut telah mencapai goal. Jika belum mencapai goal, maka node akan:

- Ditandai sebagai visited untuk menghindari pengulangan eksplorasi
- Menghasilkan semua kemungkinan neighbor (pergerakan U, D, L, R)

Untuk setiap neighbor yang dihasilkan akan dihitung:

- $G \text{ baru} = G \text{ sebelumnya} + \text{cost pergerakan}$
- $F = G$ (sama seperti UCS dan BFS)
- Neighbor dimasukkan ke stack menggunakan push() selama belum pernah dikunjungi

Proses berulang hingga solusi ditemukan atau semua kemungkinan dieksplorasi.

BAB II

ANALISIS ALGORITMA

2.1 Analisis Algoritma *PathFinding*

2.1.1 Analisis Algoritma UCS

Pada algoritma Uniform Cost Search (UCS), fungsi evaluasi ditentukan hanya berdasarkan biaya aktual yang telah ditempuh. Nilai $g(n)$ didefinisikan sebagai total cost dari state awal (Z) menuju state ke-n. Nilai ini merepresentasikan akumulasi biaya pergerakan yang telah dilakukan, yang dalam implementasi dihitung menggunakan rumus $\text{newG} = \text{cur.getG}() + \text{costStep}$ pada method `slide()`. Setiap pergerakan (atas, bawah, kiri, kanan) akan menambah nilai $g(n)$ sesuai dengan cost tile yang dilewati.

Sementara itu, nilai $f(n)$ pada UCS didefinisikan sebagai $f(n) = g(n)$. Artinya, algoritma tidak menggunakan heuristic sama sekali dan hanya mempertimbangkan biaya aktual. Hal ini terlihat pada implementasi dengan $\text{newF} = \text{newG}$.

Karakteristik UCS:

- Menghasilkan solusi optimal (cost minimum)
- Tidak menggunakan informasi posisi goal
- Eksplorasi cenderung luas

Kompleksitas:

- Waktu: $O(b^{(1+\lceil C^*/\epsilon \rceil)})$
- Ruang: $O(b^{(1+\lceil C^*/\epsilon \rceil)})$

dengan b adalah branching factor, C^* adalah cost optimal, dan ϵ adalah cost minimum per langkah.

2.1.2 Analisis Algoritma GBFS

Pada Greedy Best-First Search (GBFS), algoritma lebih berfokus pada estimasi jarak ke goal dibandingkan biaya yang sudah ditempuh. Nilai $g(n)$ tetap dihitung dengan $\text{newG} = \text{cur.getG}() + \text{costStep}$, namun tidak digunakan dalam penentuan prioritas.

Sebaliknya, algoritma menggunakan $h(n)$ sebagai heuristic, yang merupakan estimasi cost dari state saat ini menuju goal. Dalam implementasi, heuristic dihitung menggunakan Manhattan distance terhadap target berikutnya. Nilai evaluasi pada GBFS adalah $f(n) = h(n)$. Sehingga node yang dipilih adalah node yang “terlihat paling dekat” ke goal, tanpa mempertimbangkan biaya aktual.

Karakteristik GBFS:

- Tidak menjamin solusi optimal
- Lebih cepat dibanding UCS
- Sangat bergantung pada kualitas heuristic

Kompleksitas:

- Waktu: $O(b^m)$
- Ruang: $O(b^m)$

dengan m adalah kedalaman maksimum pencarian.

2.1.3 Analisis Algoritma A*

Dalam implementasi, hal ini dituliskan sebagai $\text{newF} = \text{newG} + \text{newH}$. Pendekatan ini membuat A* lebih seimbang karena tidak hanya melihat masa lalu (cost), tetapi juga memperkirakan masa depan (heuristic).

Karakteristik A*:

- Optimal jika heuristic admissible
- Lebih efisien dibanding UCS
- Performa sangat dipengaruhi kualitas heuristic

Kompleksitas:

- Waktu: $O(b^d)$
- Ruang: $O(b^d)$

dengan d adalah kedalaman solusi.

Dalam analisis heuristic, suatu heuristic dikatakan admissible jika nilainya tidak pernah melebihi cost sebenarnya menuju goal. Dengan kata lain, heuristic harus selalu menjadi lower bound dari cost optimal. Dalam implementasi ini:

- H1 (Manhattan $\times$ minCost) Admissible. Karena Manhattan distance adalah estimasi minimum jumlah langkah, dan dikalikan cost minimum.
- H2 (Euclidean Distance $\times$ minCost) Admissible. Karena *euclidean distance* merupakan estimasi jarak lurus (garis terpendek) antara posisi saat ini dengan target, jarak Euclidean tidak akan melebihi jarak sebenarnya yang harus ditempuh. Nilai *euclidean distance* $\leq$ *manhattan distance*, maka dari itu pasti Admissible.
- H3 ((Manhattan + jarak ke goal O) $\times$ minCost) Bergantung kondisi Bisa lebih akurat (informed), tetapi harus tetap tidak melebihi cost sebenarnya. Heuristik ini memang cukup bagus karena mempertimbangkan banyak kasus, tapi belum tentu Admissible, maka dari itu ditempatkan sebagai heuristik no-3.

Jika heuristic yang digunakan memenuhi sifat admissible, maka terdapat beberapa konsekuensi penting:

- A* dijamin menemukan solusi optimal
- A* tidak akan melewatkan solusi terbaik
- Proses pencarian menjadi lebih terarah tanpa mengorbankan optimalitas

2.1.4 Analisis Algoritma BFS

Pada Breadth-First Search (BFS), pencarian dilakukan secara level-by-level. Nilai $g(n)$ merepresentasikan jumlah langkah dari state awal, yang secara implisit tersimpan dalam panjang langkah (steps).

Fungsi evaluasi dapat dianggap $f(n) = g(n)$. Namun, berbeda dengan UCS, BFS menggunakan struktur Queue (FIFO), sehingga eksplorasi dilakukan berdasarkan kedalaman, bukan cost.

Karakteristik BFS:

- Menjamin solusi dengan jumlah langkah minimum
- Tidak mempertimbangkan cost tiap langkah
- Eksplorasi melebar (level-wise)

Kompleksitas:

- Waktu: $O(b^d)$
- Ruang: $O(b^d)$

2.1.5 Analisis Algoritma DFS

Pada Depth-First Search (DFS), pencarian dilakukan dengan menjelajahi satu cabang sedalam mungkin sebelum kembali (backtracking). Nilai $g(n)$ tetap merepresentasikan jumlah langkah, tetapi tidak digunakan dalam prioritas.

Fungsi evaluasi $f(n) = g(n)$ (implisit) DFS menggunakan struktur Stack (LIFO), sehingga node terakhir yang dimasukkan akan diproses terlebih dahulu.

Karakteristik DFS:

- Tidak menjamin solusi optimal
- Lebih hemat memori dibanding BFS
- Berpotensi masuk ke jalur sangat dalam atau loop

Kompleksitas:

- Waktu: $O(b^m)$
- Ruang: $O(b^m)$

2.1.6 Perbandingan Algoritma PathFinding

Untuk memahami perbedaan karakteristik masing-masing algoritma pathfinding, diperlukan analisis komparatif berdasarkan beberapa aspek penting seperti struktur data yang digunakan, fungsi prioritas, optimalitas, kompleksitas, serta performa secara umum. Perbandingan ini bertujuan untuk memberikan gambaran menyeluruh mengenai kelebihan dan kekurangan setiap algoritma sehingga dapat dipilih metode yang paling sesuai dengan kebutuhan permasalahan.

Tabel 2.6.1 Perbandingan Algoritma PathFinding (UCS, GBFS, A*, BFS, DFS)

Algoritma	UCS	GBFS	A*	BFS	DFS
Data Structure	Priority Queue	Priority Queue	Priority Queue	Queue (FIFO)	Stack (LIFO)
Prioritas $f(n)$	$g(n)$	$h(n)$	$g(n) + h(n)$	$g(n)$ level	$g(n)$ depth
Optimal?	Ya	Tidak	Ya (Dengan syarat heuristic admissible)	Step min	Tidak
Complete?	Ya	Ya	Ya (Dengan syarat heuristic admissible)	Ya	Tidak
Time Complexity	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	$O(b^m)$	$O(b^d)$	$O(b^d)$	$O(b^m)$
Space Complexity	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	$O(b^m)$	$O(b^d)$	$O(b^d)$	$O(bm)$
Mempertimbangkan Cost	Ya	Tidak	Ya	Tidak	Tidak
Mempertimbangkan Heuristic	Tidak	Ya	Ya	Tidak	Tidak
Kecepatan	Sedang	Cepat	Sangat Cepat	Cepat	Cepat
Keandalan	Tinggi	Rendah	Tinggi	Sedang	Rendah

Berdasarkan tabel perbandingan di atas, dapat disimpulkan bahwa setiap algoritma memiliki trade-off antara optimalitas, kecepatan, dan penggunaan memori. UCS unggul dalam menjamin solusi optimal berbasis cost, tetapi cenderung lebih lambat. GBFS menawarkan kecepatan tinggi dengan memanfaatkan heuristic, namun tidak menjamin solusi terbaik.

A* menjadi pilihan paling seimbang karena menggabungkan keunggulan UCS dan GBFS, yaitu tetap optimal (dengan syarat heuristic admissible) sekaligus lebih efisien. Sementara itu, BFS cocok untuk mencari solusi dengan jumlah langkah minimum tanpa mempertimbangkan cost, dan DFS lebih hemat memori tetapi memiliki risiko tidak menemukan solusi optimal maupun terjebak dalam pencarian yang terlalu dalam.

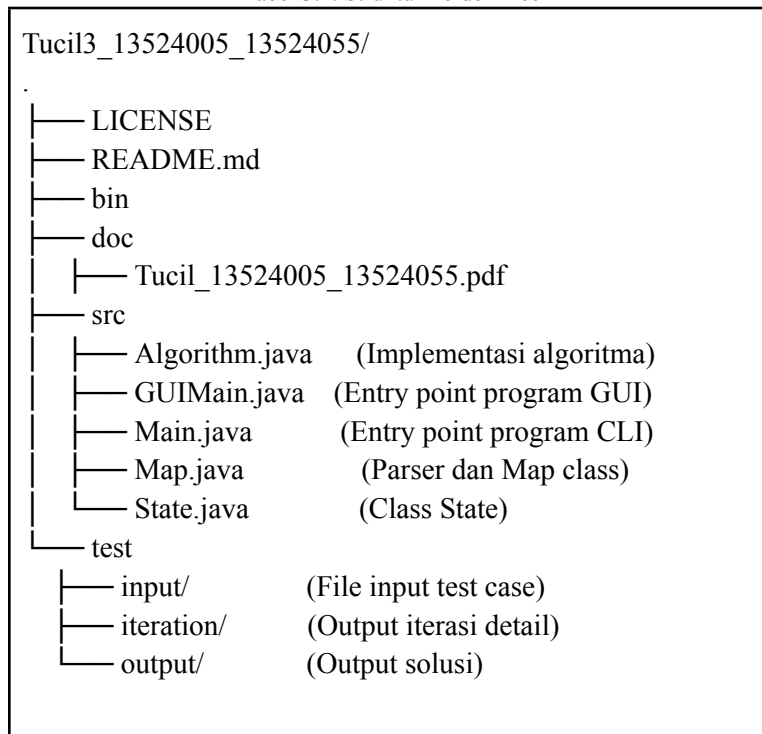
BAB III

SOURCE PROGRAM

3.1 Struktur Folder

Sebelum masuk ke dalam pembahasan source program aplikasi pathfinding ini, penting untuk memahami bagaimana struktur proyek disusun. Struktur direktori yang rapi akan mempermudah dalam memahami alur program, pembagian tanggung jawab setiap file, serta memudahkan proses pengembangan dan debugging. Berikut adalah struktur program dalam bentuk tree:

Tabel 3.1. Struktur Folder Tree



Berdasarkan struktur tersebut, dapat dilihat bahwa program dipisahkan ke dalam beberapa komponen utama yang memiliki tanggung jawab masing-masing. Folder src berisi seluruh implementasi kode program, sedangkan folder test digunakan untuk menyimpan berbagai skenario pengujian beserta hasilnya. Dengan pemisahan ini, program menjadi lebih modular, terorganisir, serta memudahkan dalam proses pengembangan, pengujian, dan analisis program.

Selanjutnya, akan ditampilkan source code dari aplikasi pathfinding yang telah diimplementasikan. Source code ini mencakup bagian utama program, representasi state, serta implementasi algoritma yang digunakan dalam proses pencarian jalur.

3.2 Source Program

3.2.1 Main.java

```
import java.io.*;
import java.util.Scanner;

public class Main{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        while (true) {
            Map map = new Map();
            System.out.println(">> Masukkan file input (EXIT untuk keluar): ");
            String file = sc.nextLine().trim();
            if (file.toLowerCase().equalsIgnoreCase("exit")) break;
            String inputName = new File(file).getName();
            String inputPath = "../test/input/" + inputName;
            if (!map.parseFile(inputPath)) return;

            System.out.println(">> Algoritma apa yang anda pilih?
(UCS/GBFS/A*/BFS/DFS)");
            String algorithm = sc.nextLine().toUpperCase();

            String hrstk= "H1";
            if (algorithm.equals("GBFS") || algorithm.equals("A*")) {
                System.out.println(">> Heuristic apa yang anda pilih? (H1/H2/H3)"
);
                hrstk = sc.nextLine().toUpperCase();
            }
            Algorithm solve = new Algorithm(map);
            solve.setHeuristicType(hrstk);
            File iterationDir = new File("../test/iteration");
            iterationDir.mkdirs();
            String iterationPath = new File(iterationDir, inputName).getPath();
            solve.setIterationFilePath(iterationPath);
            long start = System.currentTimeMillis();
            State goal = solve.solve(algorithm);
            long end = System.currentTimeMillis();

            System.out.println();
            if (goal!=null){
```

```

        String steps = goal.getSteps();
        System.out.println("Solusi yang ditemukan : " + steps);
        System.out.println("Cost dari Solusi : " + goal.getG());
        System.out.println();

        System.out.println("Initial");
        solve.printBoard(solve.loadSteps(0, steps));
        for (int i = 1; i <= steps.length(); i++){
            System.out.println("Step " + i + " : " + steps.charAt(i-1));
            solve.printBoard(solve.loadSteps(i, steps));
        }
        System.out.println(">> Waktu eksekusi: " + (end-start) + " ms");
        System.out.println(">> Banyak iterasi yang dilakukan: "+
solve.getTotalIteration() + " iterasi");

        System.out.println(">> Apakah Anda ingin melakukan playback?
(Ya/Tidak) :");
        String playbackChoice = sc.nextLine().toUpperCase();
        if (playbackChoice.equals("YA")){
            playback(sc, solve, steps);
        }

        System.out.println(">> Apakah Anda ingin menyimpan solusi?
(Ya/Tidak) : ");
        if (sc.nextLine().equalsIgnoreCase("Ya")){
            File outputDir = new File("../test/output");
            outputDir.mkdirs();
            String outputPath = new File(outputDir, inputName).getPath();
            save(outputPath, goal, solve);
        }
        } else {
            System.out.println("Solusi tidak ditemukan!");
            System.out.println("Anda bisa melihat iterasi yang dilakukan di
file test/iteration/" + inputName);
        }
    }
    sc.close();
}

private static void playback(Scanner sc, Algorithm solve, String steps){
    int curStep = 0;

```



```

        int maxStep = steps.length();
        boolean flowing = true; // Ini buat nandain kalau masih ngelakuin
        playback

        System.out.println("=== MODE PLAYBACK ===");
        System.out.println("[<] Mundur | [>] Maju | [ESC] Lompat | [Q] Keluar
        (Jangan lupa gunakan Enter)");

        while(flowing){
            System.out.println("Step " + curStep + " :");
            solve.printBoard(solve.loadSteps(curStep, steps));
            System.out.print("Input (< / > / ESC / Q) : ");
            String input = sc.nextLine().toUpperCase();
            switch(input){
                case ">" :
                    if (curStep < maxStep) curStep++;
                    break;
                case "<":
                    if (curStep > 0) curStep--;
                    break;
                case "ESC":
                    System.out.println("Pada step berapa anda ingin melakukan
                    playback :");

                    if (sc.hasNextInt()){
                        int inputStep = sc.nextInt();
                        if (inputStep >=0 && inputStep <= maxStep){
                            curStep = inputStep;
                        }
                    }
                    sc.nextLine();
                    break;
                case "Q":
                    flowing = false;
                    break;
            }
        }
    }

    private static void save(String path, State goal, Algorithm solve){
        try (PrintWriter wr = new PrintWriter(new FileWriter(path))){
            wr.println("Solusi : " + goal.getSteps());
        }
    }
}

```

```

        wr.println("Cost : " + goal.getG());
        wr.println("Initial");
        for (char[] row : solve.loadSteps(0,goal.getSteps())){
            wr.println(new String(row));
        }
        wr.println();
        for (int i = 1; i <= goal.getSteps().length();i++){
            wr.println("Step " + i + " : " + goal.getSteps().charAt(i-1));
            for (char[] row : solve.loadSteps(i,goal.getSteps())){
                wr.println(new String(row));
            }
            wr.println();
        }
        System.out.println(">> Disimpan pada " + path);
    } catch(IOException e){
        System.out.println("Gagal menyimpan file.");
    }
}
}

```

3.2.2 Algorithm.java

```

import java.util.*;

import java.io.*;

public class Algorithm{
    private Map map;
    private int minCost = Integer.MAX_VALUE;
    private int totalIteration = 0;
    private String iterationFilePath = "iteration.txt";
    private String heuristicType = "H1";
    private int[] dx = {-1,1,0,0};
    private int[] dy = {0,0,-1,1};
    private char[] dir = {'U', 'D', 'L', 'R'};

    public Algorithm(Map map){
        this.map = map;
        int[][] costs = map.getBoardCosts();
        for (int i=0; i<map.getRowCount(); i++){
            for (int j = 0; j < map.getColCount(); j++){

```

```

        if (costs[i][j] < minCost && costs[i][j] >0){
            minCost= costs[i][j];
        }
    }
}

public void setIterationFilePath(String iterationFilePath){
    if (iterationFilePath != null && !iterationFilePath.isEmpty()){
        this.iterationFilePath = iterationFilePath;
    }
}

public void setHeuristicType(String heuristicType){
    if (heuristicType != null && !heuristicType.isEmpty()){
        this.heuristicType = heuristicType;
    }
}

private int heuristicCost(int x, int y, int targetDigit, String
algorithm){
    if (algorithm.equals("UCS")) return 0;
    int targetX = map.getTargetX();
    int targetY = map.getTargetY();

    if (targetDigit <= map.getMaxDigit()){
        char[][] board = map.getBoard();
        for (int i = 0; i < map.getRowCount(); i++){
            for (int j = 0; j < map.getColCount(); j++){
                if (board[i][j] >= '0' && board[i][j] <= '9') {
                    int digit = board[i][j] - '0';
                    if (digit == targetDigit) {
                        targetX = i;
                        targetY = j;
                        break;
                    }
                }
            }
        }
    }

    int manhattan = Math.abs(x-targetX) + Math.abs(y-targetY);
    switch (heuristicType){

```

```

        case "H2":
            int dx = x-targetX;
            int dy = y-targetY;
            double euclidean = Math.sqrt(dx*dx + dy*dy);
            return (int) (euclidean * minCost);
        case "H3":
            return (manhattan + Math.abs(targetX - map.getTargetX()) +
Math.abs(targetY - map.getTargetY())) * minCost;
        case "H1":
        default:
            return manhattan * minCost;
    }
}

private State slide(State cur, int dirX, int dirY, char dir, String
algorithm, PrintWriter wr){
    int x= cur.getX();
    int y = cur.getY();
    int costStep = 0;
    int curTarget = cur.getCurDigit();
    boolean moved = false;
    char[][] board = map.getBoard();
    int[][] costs = map.getBoardCosts();
    int prevTarget = curTarget;
    while (true){
        int newX= x + dirX;
        int newY = y +dirY;
        if (newX < 0 || newX >= map.getRowCount() || newY < 0 || newY >=
map.getColCount()){
            wr.println("    -> " + dir + " GAME OVER (Keluar dari batas
papan)!");
            return null;
        }
        char tile = board[newX][newY];
        if (tile == 'X') break;
        if (tile == 'L') {
            wr.println("    -> " + dir + " GAME OVER (Melewati Lava)!");
            return null;
        }
        x = newX;
        y = newY;
    }
}

```

```

        costStep += costs[x][y];
        moved = true;

        if (Character.isDigit(tile)){
            int digit = Character.getNumericValue(tile);
            if (digit == curTarget){
                curTarget++;
            } else if(digit > curTarget){
                wr.println("    -> " + dir + " GAME OVER (Angka tidak
sesuai urutan)");
                return null;
            }
        }
    }
    if (!moved) return null;
    int newG = (cur.getG()) + costStep;
    int newH = 0;
    int newF = newG;

    if (algorithm.equals("BFS") || algorithm.equals("DFS")){
        newF = newG;
    } else {
        newH = heuristicCost(x, y, curTarget, algorithm);
        newF = (algorithm.equals("GBFS")) ? newH: (newG + newH);
    }

    boolean isTargetFulfilled = (curTarget > prevTarget);
    return new State(x, y, newG, newF, cur.getSteps() + dir, curTarget,
isTargetFulfilled);

}

public State solve(String algorithm) {
    if (algorithm.equals("BFS")){
        return solveBFS();
    } else if (algorithm.equals("DFS")){
        return solveDFS();
    } else {
        return solveBestFirst(algorithm);
    }
}
}

```

```

private State buildStartState(){
    int initialDigit = map.hasDigits() ? 0 : (map.getMaxDigit() + 1);
    return new State(map.getInitialX(), map.getInitialY(), 0, 0, "",
initialDigit, false);
}

private State solveBestFirst(String algorithm){
    PriorityQueue<State> queue = new PriorityQueue<>();
    boolean[][][] visited = new
boolean[map.getRowCount()][map.getColCount()][11];

    try (PrintWriter wr =new PrintWriter(new
FileWriter(iterationFilePath))){
        State start = buildStartState();
        queue.add(start);

        while(!queue.isEmpty()){
            State cur = queue.poll();
            totalIteration++;
            wr.println("=== Iteration " + totalIteration + " ===");
            wr.println("Initial");
            for (char[] row : loadSteps(0, cur.getSteps())){
                wr.println(new String(row));
            }
            wr.println();
            for (int i = 1; i <= cur.getSteps().length(); i++){
                wr.println("Step " + i + " : " +
cur.getSteps().charAt(i-1));
                for (char[] row : loadSteps(i, cur.getSteps())){
                    wr.println(new String(row));
                }
                wr.println();
            }

            if (map.getBoard()[cur.getX()][cur.getY()] == 'O' &&
cur.getCurDigit() > map.getMaxDigit()) {
                wr.println("Found Solution");
                wr.println();
                return cur;
            }
        }
    }
}

```

```

        if (visited[cur.getX()][cur.getY()][cur.getCurDigit()])
continue;

        visited[cur.getX()][cur.getY()][cur.getCurDigit()] = true;

        for (int i = 0; i < 4; i++){
            char nextDir = dir[i];
            State nextState = slide(cur,dx[i],
dy[i],nextDir,algorithm,wr);
            if (nextState != null &&
!visited[nextState.getX()][nextState.getY()][nextState.getCurDigit()]){
                queue.add(nextState);
            }
        }
        wr.println();
    }
    } catch (IOException e){
        System.out.println("Gagal menulis iterasi ke file iterasi.txt");
    }
    return null;
}

private State solveBFS(){
    Queue<State> queue = new LinkedList<>();
    boolean[][][] visited = new
boolean[map.getRowCount()][map.getColCount()][11];

    try (PrintWriter wr =new PrintWriter(new
FileWriter(iterationFilePath))){
        State start = buildStartState();
        queue.add(start);

        while(!queue.isEmpty()){
            State cur = queue.poll();
            totalIteration++;
            wr.println("=== Iteration " + totalIteration + " ===");
            wr.println("Initial");
            for (char[] row : loadSteps(0, cur.getSteps())){
                wr.println(new String(row));
            }
            wr.println();

```

```

        for (int i = 1; i <= cur.getSteps().length(); i++){
            wr.println("Step " + i + " : " +
cur.getSteps().charAt(i-1));
            for (char[] row : loadSteps(i, cur.getSteps())){
                wr.println(new String(row));
            }
            wr.println();
        }

        if (map.getBoard()[cur.getX()][cur.getY()] == 'O' &&
cur.getCurDigit() > map.getMaxDigit()) {
            wr.println("Found Solution");
            wr.println();
            return cur;
        }

        if (visited[cur.getX()][cur.getY()][cur.getCurDigit()])
continue;

        visited[cur.getX()][cur.getY()][cur.getCurDigit()] = true;

        for (int i = 0; i < 4; i++){
            char nextDir = dir[i];
            State nextState = slide(cur,dx[i],
dy[i],nextDir,"BFS",wr);
            if (nextState != null &&
!visited[nextState.getX()][nextState.getY()][nextState.getCurDigit()]){
                queue.add(nextState);
            }
        }
        wr.println();
    }
} catch (IOException e){
    System.out.println("Gagal menulis iterasi ke file iterasi.txt");
}
return null;
}

private State solveDFS(){
    Deque<State> stack = new ArrayDeque<>();
    boolean[][][] visited = new
boolean[map.getRowCount()][map.getColCount()][11];

```



```

        try (PrintWriter wr = new PrintWriter(new
FileWriter(iterationFilePath))) {
            State start = buildStartState();
            stack.push(start);

            while (!stack.isEmpty()) {
                State cur = stack.pop();
                totalIteration++;
                wr.println("=== Iteration " + totalIteration + " ===");
                wr.println("Initial");
                for (char[] row : loadSteps(0, cur.getSteps())) {
                    wr.println(new String(row));
                }
                wr.println();
                for (int i = 1; i <= cur.getSteps().length(); i++) {
                    wr.println("Step " + i + " : " +
cur.getSteps().charAt(i-1));
                    for (char[] row : loadSteps(i, cur.getSteps())) {
                        wr.println(new String(row));
                    }
                    wr.println();
                }

                if (map.getBoard()[cur.getX()][cur.getY()] == 'O' &&
cur.getCurDigit() > map.getMaxDigit()) {
                    wr.println("Found Solution");
                    wr.println();
                    return cur;
                }

                if (visited[cur.getX()][cur.getY()][cur.getCurDigit()])
continue;
                visited[cur.getX()][cur.getY()][cur.getCurDigit()] = true;

                for (int i = 0; i < 4; i++) {
                    char nextDir = dir[i];
                    State nextState = slide(cur, dx[i],
dy[i], nextDir, "DFS", wr);
                    if (nextState != null &&
!visited[nextState.getX()][nextState.getY()][nextState.getCurDigit()]) {

```

```

        stack.push(nextState);
    }
}
wr.println();
}
} catch (IOException e){
    System.out.println("Gagal menulis iterasi ke file iterasi.txt");
}
return null;
}

public char[][] loadSteps(int n, String steps){
    char[][] board = new char[map.getRowCount()][map.getColCount()];
    for (int i = 0; i < map.getRowCount(); i++){
        for (int j = 0; j < map.getColCount(); j++){
            board[i][j] = map.getBoard()[i][j];
        }
    }
    int curX = map.getInitialX();
    int curY = map.getInitialY();
    board[curX][curY] = 'Z';

    for(int i = 0; i < n; i++){
        char move = steps.charAt(i);
        int dX = 0;
        int dY = 0;
        if(move == 'U') dX = -1;
        else if(move == 'D') dX = 1;
        else if(move == 'L') dY = -1;
        else if(move == 'R') dY = 1;

        while (true){
            int newX = curX + dX;
            int newY = curY + dY;
            if (board[newX][newY] == 'X')break;

            char oriTile = map.getBoard()[curX][curY];
            if (Character.isDigit(oriTile)) {
                board[curX][curY] = oriTile;
            } else if (oriTile != 'O' && oriTile != 'X' && oriTile != 'L')
        {

```

```

        board[curX][curY] = '*';
    } else {
        board[curX][curY] = oriTile;
    }

    curX = newX;
    curY = newY;
}

    board[curX][curY] = 'Z';
}
return board;
}

public void printBoard(char[][] board) {
    for (char[] row : board) {
        System.out.println(new String(row));
    }
    System.out.println();
}

public int getTotalIteration() {return totalIteration;}
}

```

3.2.3 State.java

```

public class State implements Comparable<State>{
    private int x, y;
    private int g;
    private int f;
    private String steps;
    private int curDigit;
    private boolean usedTargetLastMove;

    public State(int x, int y, int g, int f, String steps, int curDigit,
boolean usedTargetLastMove) {
        this.x = x;
        this.y = y;
        this.g = g;
    }
}

```

```

        this.f = f;
        this.steps = steps;
        this.curDigit = curDigit;
        this.usedTargetLastMove = usedTargetLastMove;
    }

    public int getX(){ return x;}
    public int getY(){ return y;}
    public int getG(){ return g;}
    public int getF(){ return f;}
    public String getSteps(){ return steps;}
    public int getCurDigit(){ return curDigit;}
    public boolean getUsedTargetLastMove(){ return usedTargetLastMove;}

    public int compareTo(State other){
        return Integer.compare(this.f, other.f);
    }
}

```

3.2.4 Map.java

```

import java.io.File;
import java.util.Scanner;
import java.io.FileNotFoundException;

public class Map {
    private int N, M;
    private char[][] board;
    private int[][] boardCosts;
    private int maxDigit;
    private int initX, initY;
    private int targetX, targetY;
    private boolean hasDigit;

    public boolean parseFile(String filepath){
        try {
            Scanner sc = new Scanner(new File(filepath));
            if (!sc.hasNextInt()){
                System.out.println("[ERROR] File kosong atau tidak memiliki

```

```

penentu dimensi N M!");
    }
    N = sc.nextInt();
    M = sc.nextInt();
    board = new char[N][M];
    boardCosts = new int[N][M];
    int countZ = 0, countO = 0;
    boolean[] digitInBoard = new boolean[10];

    for (int i = 0; i < N; i++){
        if(!sc.hasNext()){
            System.out.println("[ERROR] Jumlah baris papan kurang dari
" + N + "! Ngitung dong atleast :(");
            return false;
        }
        String line = sc.next();
        if (line.length() != M){
            System.out.println("[ERROR] Panjang baris ke-" + (i+1) + "
tidak sama dengan M! Ngitung dong atleast :(");
            return false;
        }
        board[i] = line.toCharArray();

        for (int j = 0; j<M; j++){
            char c = board[i][j];
            if (c == 'Z') {
                countZ++;
                initX = i;
                initY = j;
            }
            else if (c == 'O') {
                countO++;
                targetX = i;
                targetY = j;
            }
            else if (Character.isDigit(c)){
                int digit = Character.getNumericValue(c);
                digitInBoard[digit] = true;
                hasDigit = true;
                if (digit > maxDigit){
                    maxDigit = digit;
                }
            }
        }
    }
}

```

```

    }
    } else if (c != 'X' && c != '*' && c != 'L'){
        System.out.println("[ERROR] Karakter tidak dikenal '"
+ c + "' di papan! Ngetik yang bener dong :(");
        return false;
    }

    }
}
if (countZ != 1){
    System.out.println("[ERROR] Papan harus memiliki tepat satu Z.
Terdapat " + countZ + " di papan! Fokus ke satu player dong :(");
    return false;
}
if (countO != 1){
    System.out.println("[ERROR] Papan harus memiliki tepat satu O.
Terdapat " + countO + " di papan! Sama seperti di kehidupan kita hanya boleh
punya satu pacar yah...");
    return false;
}
boolean foundNumberAfterGap = false;
for (int i = 0; i < 10; i++){
    if (digitInBoard[i]){
        if(foundNumberAfterGap){
            System.out.println("[ERROR] Urutan tidak valid! Tolong
ngurut dong angka di papannya...");
            return false;
        }
    } else {
        foundNumberAfterGap = true;
    }
}
for (int i = 0; i < N; i++){
    for (int j = 0; j < M ; j++){
        if (!sc.hasNextInt()){
            System.out.println("[ERROR] Data cost kurang untuk
papan berukuran N x M :(");
            return false;
        }
        boardCosts[i][j] = sc.nextInt();
    }
}

```

```

        }
        sc.close();
        return true;
    } catch (FileNotFoundException e){
        System.out.println("[ERROR] File " + filepath + " tidak
ditemukan!");
        return false;
    }
}

public int getRowCount(){ return N;}
public int getColCount(){ return M;}
public char[][] getBoard(){ return board;}
public int[][] getBoardCosts(){ return boardCosts;}
public int getMaxDigit(){ return maxDigit;}
public boolean hasDigits(){ return hasDigit;}
public int getInitialX(){ return initX;}
public int getInitialY(){ return initY;}
public int getTargetX(){ return targetX;}
public int getTargetY(){ return targetY;}
}

```

3.2.5 GUIMain.java

```

import javax.swing.*;
import javax.swing.filechooser.FileNameExtensionFilter;
import java.awt.*;
import java.awt.event.*;
import java.io.File;

public class GUIMain extends JFrame {
    private Map map;
    private Algorithm algorithm;
    private State solution;
    private String currentAlgorithm = "UCS";
    private String currentHeuristic = "H1";
    private int currentStep = 0;
    private long executionTime;
    private Timer animationTimer;
    private int animationFromStep = 0;
}

```

```

private int animationToStep = 0;
private float animationProgress = 1f;

private static final int ANIMATION_DURATION_MS = 180;
private static final int ANIMATION_FRAME_MS = 16;
private JPanel boardPanel;
private JComboBox<String> algorithmSelector;
private JComboBox<String> heuristicSelector;
private JButton loadButton, solveButton, saveButton;
private JLabel fileLabel, costLabel, iterLabel, timeLabel, stepLabel;
private JButton prevButton, nextButton;
private JLabel solutionLabel;

private static final int CELL_SIZE = 40;
private static final int MIN_CELL_SIZE = 14;

// Minimal Modern Color Palette
private static final Color BG_COLOR = new Color(250, 250, 250); //
Off-white
private static final Color PANEL_BG = new Color(245, 245, 245); //
Light gray
private static final Color ACCENT_COLOR = new Color(0, 120, 150); //
Teal accent
private static final Color TEXT_PRIMARY = new Color(30, 30, 30); //
Dark text
private static final Color TEXT_SECONDARY = new Color(100, 100, 100); //
Gray text
private static final Color BORDER_COLOR = new Color(220, 220, 220); //
Subtle border

public GUIMain() {
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception e) {
        e.printStackTrace();
    }

    setTitle("TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM");
    setSize(1000, 700);
    setLocationRelativeTo(null);
}

```



```

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setResizable(true);

JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
mainPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
mainPanel.setBackground(BG_COLOR);

boardPanel = new JPanel() {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        drawBoard(g);
    }
};

boardPanel.setPreferredSize(new Dimension(500, 600));
boardPanel.setBackground(Color.WHITE);
boardPanel.setBorder(BorderFactory.createLineBorder(BORDER_COLOR, 1));
boardPanel.setFocusable(true);

boardPanel.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        boardPanel.requestFocusInWindow();
    }
});

boardPanel.addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        handleBoardKeyPress(e);
    }
});

JPanel controlPanel = new JPanel();
controlPanel.setLayout(new BoxLayout(controlPanel, BoxLayout.Y_AXIS));
controlPanel.setPreferredSize(new Dimension(450, 600));
controlPanel.setBackground(BG_COLOR);

JPanel filePanel = createFilePanel();
controlPanel.add(filePanel);
controlPanel.add(Box.createVerticalStrut(15));

```

```

    JPanel algoPanel = createAlgoritmPanel();
    controlPanel.add(algoPanel);
    controlPanel.add(Box.createVerticalStrut(15));

    JPanel solvePanel = createSolvePanel();
    controlPanel.add(solvePanel);
    controlPanel.add(Box.createVerticalStrut(15));

    JPanel infoPanel = createInfoPanel();
    controlPanel.add(infoPanel);
    controlPanel.add(Box.createVerticalStrut(15));

    JPanel playbackPanel = createPlaybackPanel();
    controlPanel.add(playbackPanel);
    controlPanel.add(Box.createVerticalStrut(15));

    JPanel savePanel = new JPanel();
    savePanel.setLayout(new FlowLayout(FlowLayout.LEFT));
    savePanel.setBackground(BG_COLOR);

    saveButton = new JButton("SAVE");
    saveButton.setEnabled(false);
    saveButton.setFont(new Font("Segoe UI", Font.BOLD, 11));
    saveButton.setBackground(ACCENT_COLOR);
    saveButton.setForeground(Color.WHITE);
    saveButton.setFocusPainted(false);
    saveButton.setBorder(BorderFactory.createEmptyBorder(6, 15, 6, 15));
    saveButton.addActionListener(e -> saveResult());
    savePanel.add(saveButton);
    controlPanel.add(savePanel);

    controlPanel.add(Box.createVerticalGlue());

    mainPanel.add(new JScrollPane(boardPanel), BorderLayout.CENTER);
    mainPanel.add(controlPanel, BorderLayout.EAST);

    add(mainPanel);
    setVisible(true);
}

```

```

private JPanel createFilePanel() {
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
    panel.setBackground(PANEL_BG);
    panel.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(BORDER_COLOR, 1),
        BorderFactory.createEmptyBorder(10, 10, 10, 10)
    ));

    fileLabel = new JLabel("No file loaded.");
    fileLabel.setFont(new Font("Segoe UI", Font.PLAIN, 12));
    fileLabel.setForeground(TEXT_SECONDARY);

    loadButton = new JButton("LOAD FILE");
    loadButton.setFont(new Font("Segoe UI", Font.BOLD, 11));
    loadButton.setBackground(ACCENT_COLOR);
    loadButton.setForeground(Color.WHITE);
    loadButton.setFocusPainted(false);
    loadButton.setBorder(BorderFactory.createEmptyBorder(8, 15, 8, 15));
    loadButton.addActionListener(e -> loadFile());

    panel.add(fileLabel);
    panel.add(Box.createVerticalStrut(10));
    panel.add(loadButton);

    return panel;
}

private JPanel createAlgorithmPanel() {
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
    panel.setBackground(PANEL_BG);
    panel.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(BORDER_COLOR, 1),
        BorderFactory.createEmptyBorder(10, 10, 10, 10)
    ));

    algoLabel = new JLabel("Select Algorithm:");
    algoLabel.setFont(new Font("Segoe UI", Font.BOLD, 12));
    algoLabel.setForeground(TEXT_PRIMARY);

```

```

        algorithmSelector = new JComboBox<>(new String[]{"UCS", "GBFS", "A*",
"BFS", "DFS"});
        algorithmSelector.setMaximumSize(new Dimension(Integer.MAX_VALUE,
30));
        algorithmSelector.setFont(new Font("Segoe UI", Font.PLAIN, 11));
        algorithmSelector.addActionListener(e -> {
            currentAlgorithm = (String) algorithmSelector.getSelectedItem();
            boolean useHeuristic = !currentAlgorithm.equals("UCS") &&
                                !currentAlgorithm.equals("BFS") &&
                                !currentAlgorithm.equals("DFS");
            heuristicSelector.setEnabled(useHeuristic);
        });

        JLabel heuristicLabel = new JLabel("Select Heuristic:");
        heuristicLabel.setFont(new Font("Segoe UI", Font.BOLD, 12));
        heuristicLabel.setForeground(TEXT_PRIMARY);

        heuristicSelector = new JComboBox<>(new String[]{"H1", "H2", "H3"});
        heuristicSelector.setMaximumSize(new Dimension(Integer.MAX_VALUE,
30));
        heuristicSelector.setFont(new Font("Segoe UI", Font.PLAIN, 11));
        heuristicSelector.addActionListener(e -> currentHeuristic = (String)
heuristicSelector.getSelectedItem());

        panel.add(algoLabel);
        panel.add(algorithmSelector);
        panel.add(Box.createVerticalStrut(10));
        panel.add(heuristicLabel);
        panel.add(heuristicSelector);

        return panel;
    }

    private JPanel createSolvePanel(){
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        panel.setBackground(PANEL_BG);
        panel.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(BORDER_COLOR, 1),
            BorderFactory.createEmptyBorder(10, 10, 10, 10)
        ));
    }

```

```

        solveButton = new JButton("SOLVE");
        solveButton.setFont(new Font("Segoe UI", Font.BOLD, 14));
        solveButton.setEnabled(false);
        solveButton.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));
        solveButton.setBackground(ACCENT_COLOR);
        solveButton.setForeground(Color.WHITE);
        solveButton.setFocusPainted(false);
        solveButton.setBorderPainted(false);
        solveButton.addActionListener(e -> solve());

        panel.add(solveButton);
        return panel;
    }

    private JPanel createInfoPanel(){
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        panel.setBackground(PANEL_BG);
        panel.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(BORDER_COLOR, 1),
            BorderFactory.createEmptyBorder(10, 10, 10, 10)
        ));

        JLabel titleLabel = new JLabel("Result Info");
        titleLabel.setFont(new Font("Segoe UI", Font.BOLD, 12));
        titleLabel.setForeground(TEXT_PRIMARY);

        solutionLabel = new JLabel("Solution: -");
        costLabel = new JLabel("Cost: -");
        iterLabel = new JLabel("Iterations: -");
        timeLabel = new JLabel("Execution Time: - ms");
        stepLabel = new JLabel("Step: 0/?");

        Font infoFont = new Font("Segoe UI", Font.PLAIN, 11);
        Color infoColor = TEXT_SECONDARY;

        solutionLabel.setFont(new Font("Monospaced", Font.PLAIN, 10));
        solutionLabel.setForeground(infoColor);
        costLabel.setFont(infoFont);
        costLabel.setForeground(infoColor);

```

```

        iterLabel.setFont(infoFont);
        iterLabel.setForeground(infoColor);
        timeLabel.setFont(infoFont);
        timeLabel.setForeground(infoColor);
        stepLabel.setFont(infoFont);
        stepLabel.setForeground(infoColor);

        panel.add(titleLabel);
        panel.add(Box.createVerticalStrut(8));
        panel.add(solutionLabel);
        panel.add(Box.createVerticalStrut(5));
        panel.add(costLabel);
        panel.add(Box.createVerticalStrut(5));
        panel.add(iterLabel);
        panel.add(Box.createVerticalStrut(5));
        panel.add(timeLabel);
        panel.add(Box.createVerticalStrut(5));
        panel.add(stepLabel);

        return panel;
    }

    private JPanel createPlaybackPanel() {
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        panel.setBackground(PANEL_BG);
        panel.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(BORDER_COLOR, 1),
            BorderFactory.createEmptyBorder(10, 10, 10, 10)
        ));

        JLabel helpLabel = new JLabel("<html>Controls:<br>[← →]
Prev/Next<br>[ESC] Jump to step</html>");
        helpLabel.setFont(new Font("Segoe UI", Font.PLAIN, 10));
        helpLabel.setForeground(TEXT_SECONDARY);

        JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10,
0));
        buttonPanel.setBackground(PANEL_BG);

        prevButton = new JButton("< Prev");

```

```

        prevButton.setEnabled(false);
        prevButton.setFont(new Font("Segoe UI", Font.BOLD, 11));
        prevButton.setBackground(ACCENT_COLOR);
        prevButton.setForeground(Color.WHITE);
        prevButton.setFocusPainted(false);
        prevButton.setBorder(BorderFactory.createEmptyBorder(6, 12, 6, 12));
        prevButton.addActionListener(e -> previousStep());

        nextButton = new JButton("Next >");
        nextButton.setEnabled(false);
        nextButton.setFont(new Font("Segoe UI", Font.BOLD, 11));
        nextButton.setBackground(ACCENT_COLOR);
        nextButton.setForeground(Color.WHITE);
        nextButton.setFocusPainted(false);
        nextButton.setBorder(BorderFactory.createEmptyBorder(6, 12, 6, 12));
        nextButton.addActionListener(e -> nextStep());

        buttonPanel.add(prevButton);
        buttonPanel.add(nextButton);

        panel.add(helpLabel);
        panel.add(Box.createVerticalStrut(10));
        panel.add(buttonPanel);

        return panel;
    }

    private void loadFile() {
        JFileChooser chooser = new JFileChooser("../test/input");
        FileNameExtensionFilter filter = new FileNameExtensionFilter("Text
files", "txt");
        chooser.setFileFilter(filter);

        if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
            File file = chooser.getSelectedFile();
            map = new Map();
            if (map.parseFile(file.getAbsolutePath())) {
                fileLabel.setText("Loaded: " + file.getName());
                solveButton.setEnabled(true);
                resetResult();
                boardPanel.requestFocus();
            }
        }
    }

```

```

        } else {
            JOptionPane.showMessageDialog(this, "Failed to load file",
"Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

private void solve() {
    if (map == null) return;

    if (animationTimer != null && animationTimer.isRunning()) {
        animationTimer.stop();
    }

    solveButton.setEnabled(false);
    algorithm = new Algorithm(map);
    algorithm.setHeuristicType(currentHeuristic);

    File iterationDir = new File("../test/iteration");
    iterationDir.mkdirs();
    String iterationPath = new File(iterationDir,
"temp.txt").getAbsolutePath();
    algorithm.setIterationFilePath(iterationPath);

    long startTime = System.currentTimeMillis();
    solution = algorithm.solve(currentAlgorithm);
    executionTime = System.currentTimeMillis() - startTime;

    if (solution != null) {
        currentStep = 0;
        costLabel.setText("Cost: " + solution.getG());
        iterLabel.setText("Iterations: " + algorithm.getTotalIteration());
        timeLabel.setText("Time: " + executionTime + " ms");
        solutionLabel.setText("Solution: " + solution.getSteps());
        updateStepLabel();

        prevButton.setEnabled(false);
        nextButton.setEnabled(true);
        saveButton.setEnabled(true);

        updateBoard();
    }
}

```



```

        boardPanel.requestFocus();
    } else {
        JOptionPane.showMessageDialog(this, "No solution found", "Result",
JOptionPane.INFORMATION_MESSAGE);
    }

    solveButton.setEnabled(true);
}

private void nextStep() {
    if (solution != null && currentStep < solution.getSteps().length()) {
        animateToStep(currentStep + 1);
    }
}

private void previousStep() {
    if (currentStep > 0) {
        animateToStep(currentStep - 1);
    }
}

private void handleBoardKeyPress(KeyEvent e) {
    if (solution == null) return;

    switch (e.getKeyCode()) {
        case KeyEvent.VK_LEFT:
            previousStep();
            break;
        case KeyEvent.VK_RIGHT:
            nextStep();
            break;
        case KeyEvent.VK_ESCAPE:
            jumpToStep();
            break;
    }
}

private void jumpToStep() {
    if (solution == null) return;

    String input = JOptionPane.showInputDialog(this,

```

```

        "Jump to step (0-" + solution.getSteps().length() + "):",
        "0");

    if (input != null) {
        try {
            int step = Integer.parseInt(input.trim());
            if (step >= 0 && step <= solution.getSteps().length()) {
                currentStep = step;
                updateBoard();
            } else {
                JOptionPane.showMessageDialog(this,
                    "Step harus antara 0 dan " +
solution.getSteps().length(),
                    "Invalid Input",
                    JOptionPane.WARNING_MESSAGE);
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this,
                "Masukkan angka yang valid",
                "Invalid Input",
                JOptionPane.WARNING_MESSAGE);
        }
    }

    private void updateBoard() {
        prevButton.setEnabled(currentStep > 0);
        nextButton.setEnabled(currentStep < (solution != null ?
solution.getSteps().length() : 0));
        updateStepLabel();
        boardPanel.repaint();
    }

    private void updateStepLabel() {
        if (solution != null) {
            stepLabel.setText("Step: " + currentStep + "/" +
solution.getSteps().length());
        }
    }

    private void drawBoard(Graphics g) {

```

```

        if (map == null) return;

        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        char[][] board = (solution != null) ? algorithm.loadSteps(currentStep,
solution.getSteps()) : map.getBoard();

        char[][] fromBoard = null;
        char[][] toBoard = null;
        Point fromPos = null;
        Point toPos = null;

        boolean animating = animationTimer != null &&
animationTimer.isRunning();
        if (animating && solution != null) {
            fromBoard = algorithm.loadSteps(animationFromStep,
solution.getSteps());
            toBoard = algorithm.loadSteps(animationToStep,
solution.getSteps());
            fromPos = findPlayerPosition(fromBoard);
            toPos = findPlayerPosition(toBoard);
        }

        int[][] costs = map.getBoardCosts();

        int rows = map.getRowCount();
        int cols = map.getColCount();
        int cellSize = getCellSize(rows, cols);
        int boardWidth = cols * cellSize;
        int boardHeight = rows * cellSize;
        int offsetX = Math.max(0, (boardPanel.getWidth() - boardWidth) / 2);
        int offsetY = Math.max(0, (boardPanel.getHeight() - boardHeight) / 2);

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                int x = offsetX + (j * cellSize);
                int y = offsetY + (i * cellSize);
                char cell = board[i][j];
                int cost = costs[i][j];
            }
        }

```

```

        Color bgColor = getCostColor(cost);
        g2d.setColor(bgColor);
        g2d.fillRect(x, y, cellSize, cellSize);

        g2d.setColor(BORDER_COLOR);
        g2d.drawRect(x, y, cellSize, cellSize);

        if (cell == 'Z') {
            if (!animating) {
                g2d.setColor(ACCENT_COLOR);
                int pad = Math.max(2, cellSize / 6);
                g2d.fillOval(x + pad, y + pad, cellSize - (pad * 2),
cellSize - (pad * 2));
                g2d.setColor(Color.WHITE);
                g2d.setFont(new Font("Segoe UI", Font.BOLD,
Math.max(10, cellSize / 2)));
                FontMetrics fm = g2d.getFontMetrics();
                String symbol = "Z";
                int sx = x + (cellSize - fm.stringWidth(symbol)) / 2;
                int sy = y + ((cellSize - fm.getHeight()) / 2) +
fm.getAscent();
                g2d.drawString(symbol, sx, sy);
            }
        } else {
            g2d.setFont(new Font("Segoe UI", Font.BOLD, Math.max(10,
cellSize / 2)));
            g2d.setColor(getSymbolColor(cell));
            String symbol = String.valueOf(cell);
            FontMetrics fm = g2d.getFontMetrics();
            int sx = x + (cellSize - fm.stringWidth(symbol)) / 2;
            int sy = y + ((cellSize - fm.getHeight()) / 2) +
fm.getAscent();
            g2d.drawString(symbol, sx, sy);
        }
    }

    if (animating && fromPos != null && toPos != null) {
        float px = lerp(fromPos.x, toPos.x, animationProgress);
        float py = lerp(fromPos.y, toPos.y, animationProgress);
    }
}

```

```

        int zx = offsetX + Math.round(px * cellSize);
        int zy = offsetY + Math.round(py * cellSize);

        g2d.setColor(new Color(33, 150, 243));
        int pad = Math.max(2, cellSize / 6);
        g2d.fillOval(zx + pad, zy + pad, cellSize - (pad * 2), cellSize -
(pad * 2));

        g2d.setColor(Color.WHITE);
        g2d.setFont(new Font("Segoe UI", Font.BOLD, Math.max(10, cellSize
/ 2)));

        FontMetrics fm = g2d.getFontMetrics();
        String symbol = "Z";
        int sx = zx + (cellSize - fm.stringWidth(symbol)) / 2;
        int sy = zy + ((cellSize - fm.getHeight()) / 2) + fm.getAscent();
        g2d.drawString(symbol, sx, sy);
    }
}

private int getCellSize(int rows, int cols) {
    if (rows <= 0 || cols <= 0) return CELL_SIZE;
    int panelW = Math.max(1, boardPanel.getWidth());
    int panelH = Math.max(1, boardPanel.getHeight());
    int fitSize = Math.min(panelW / cols, panelH / rows);
    int size = Math.min(CELL_SIZE, fitSize);
    return Math.max(MIN_CELL_SIZE, size);
}

private Color getCostColor(int cost) {
    if (cost == 0) return new Color(240, 240, 240); // Very light gray
    // Softer gradient
    int normalized = Math.min(200, cost * 4);
    return new Color(255 - normalized/2, 200 - normalized/3, 100 +
normalized/4);
}

private Color getSymbolColor(char symbol) {
    switch (symbol) {
        case 'Z': return Color.BLUE;
        case 'O': return Color.RED;
    }
}

```

```

        case 'X': return Color.BLACK;
        case 'L': return new Color(255, 165, 0); // Orange
        case '*': return new Color(128, 128, 128); // Gray
        default:
            if (Character.isDigit(symbol)) return Color.GREEN;
            return Color.BLACK;
    }
}

private void saveResult() {
    if (solution == null) return;

    JFileChooser chooser = new JFileChooser("../test/output");
    if (chooser.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
        File file = chooser.getSelectedFile();
        try (java.io.PrintWriter wr = new java.io.PrintWriter(new
java.io.FileWriter(file))) {
            wr.println("Algoritma: " + currentAlgorithm);
            if (!currentAlgorithm.equals("UCS") &&
!currentAlgorithm.equals("BFS") && !currentAlgorithm.equals("DFS")) {
                wr.println("Heuristic: " + currentHeuristic);
            }
            wr.println("Solusi : " + solution.getSteps());
            wr.println("Cost : " + solution.getG());
            wr.println("Iterasi: " + algorithm.getTotalIteration());
            wr.println("Waktu: " + executionTime + " ms");
            wr.println();
            wr.println("Initial");
            for (char[] row : algorithm.loadSteps(0, solution.getSteps()))
            {
                wr.println(new String(row));
            }
            wr.println();
            for (int i = 1; i <= solution.getSteps().length(); i++) {
                wr.println("Step " + i + " : " +
solution.getSteps().charAt(i - 1));
                for (char[] row : algorithm.loadSteps(i,
solution.getSteps())) {
                    wr.println(new String(row));
                }
            }
            wr.println();
        }
    }
}

```

```

        }
        JOptionPane.showMessageDialog(this, "Saved to: " +
file.getAbsolutePath());
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Failed to save file",
"Error", JOptionPane.ERROR_MESSAGE);
    }
}

private void resetResult() {
    if (animationTimer != null && animationTimer.isRunning()) {
        animationTimer.stop();
    }
    solution = null;
    currentStep = 0;
    solutionLabel.setText("Solution: -");
    costLabel.setText("Cost: -");
    iterLabel.setText("Iterations: -");
    timeLabel.setText("Time: -");
    stepLabel.setText("Step: 0/?");
    prevButton.setEnabled(false);
    nextButton.setEnabled(false);
    saveButton.setEnabled(false);
    boardPanel.repaint();
}

private void animateToStep(int targetStep) {
    if (animationTimer != null && animationTimer.isRunning()) {
        animationTimer.stop();
    }

    animationFromStep = currentStep;
    animationToStep = targetStep;
    animationProgress = 0f;

    animationTimer = new Timer(ANIMATION_FRAME_MS, e -> {
        animationProgress += (float) ANIMATION_FRAME_MS /
ANIMATION_DURATION_MS;

        if (animationProgress >= 1f) {

```

```

        animationProgress = 1f;
        ((Timer) e.getSource()).stop();
        currentStep = animationToStep;
        updateBoard();
    } else {
        boardPanel.repaint();
    }
});

    animationTimer.start();
}
private Point findPlayerPosition(char[][] board) {
    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[i].length; j++) {
            if (board[i][j] == 'Z') {
                return new Point(j, i);
            }
        }
    }
    return null;
}

private float lerp(float a, float b, float t) {
    return a + (b - a) * t;
}
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new GUIMain());
}
}

```


BAB IV

MASUKAN DAN LUARAN PROGRAM

Pada bagian ini ditampilkan beberapa contoh test case yang digunakan untuk menguji kinerja aplikasi pathfinding yang telah dibuat. Setiap test case terdiri dari file input dan output yang dihasilkan oleh program. Melalui pengujian ini, dapat diamati bagaimana algoritma bekerja dalam menemukan solusi berdasarkan kondisi peta yang berbeda-beda.

4.1 Test Case 1

Papan yang sudah valid dan goal terdefinisi :

INPUT	
<pre>7 7 XXXXXX X0***X X**X**X X***0X X1***LX XZ**X*X XXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999</pre>	
OUTPUT	
	CLI

UCS

```
>> Masukkan file input:
test1.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
UCS
```

```
Solusi yang ditemukan : RULUDRUR
Cost dari Solusi : 87
```

Initial

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX
```

Step 1 : R

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX
```

Step 2 : U

```
XXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX
```

Step 8 : R

```
XXXXXXX
X0***X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXXX
```

```
>> Waktu eksekusi: 21 ms
>> Banyak iterasi yang dilakukan: 15 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ..\test\output\test1.txt
```

Step 3 : L

```
XXXXXXX
X0***X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX
```

Step 4 : U

```
XXXXXXX
XZ***X
X**X**X
X***OX
X1***LX
X***X*X
XXXXXXX
```

Step 5 : D

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX
```

Step 6 : R

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX
```

GBFS (H1)

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1
```

```
Solusi yang ditemukan : RULUDRUR
Cost dari Solusi : 87
```

Initial

```
XXXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXXX
```

Step 1 : R

```
XXXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXXX
```

Step 2 : U

```
XXXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXXX
```

Step 7 : U

```
XXXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXXX
```

Step 8 : R

```
XXXXXXXX
X0***X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXXXX
```

```
>> Waktu eksekusi: 4 ms
>> Banyak iterasi yang dilakukan: 12 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ..\test\output\test1.txt
to another file input (ENTER to skip)
```

Step 3 : L

```
XXXXXXXX
X0***X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXXX
```

Step 4 : U

```
XXXXXXXX
XZ***X
X**X**X
X***OX
X1***LX
X***X*X
XXXXXXXX
```

Step 5 : D

```
XXXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXXX
```

Step 6 : R

```
XXXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXXX
```

GBFS (H2)

```
>> Masukkan file input (EXIT untuk keluar):
test1.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H2

Solusi yang ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial
XXXXXXXX
X0***X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXXX

Step 1 : R
XXXXXXXX
X0***X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXXX

Step 2 : U
XXXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXXX

Step 3 : L
XXXXXXXX
X0***X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXXX

Step 4 : U
XXXXXXXX
XZ***X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXXX

Step 5 : D
XXXXXXXX
X0***X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXXX

Step 6 : R
XXXXXXXX
X0***X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXXX

Step 7 : U
XXXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXXX

Step 8 : R
XXXXXXXX
X0***X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXXXX

>> Waktu eksekusi: 13 ms
>> Banyak iterasi yang dilakukan: 12 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ../test/output/test1.txt
Masukkan file input (EXIT untuk keluar):
```

GBFS (H3)

```
>> Masukkan file input (EXIT untuk keluar):
test1.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H3

Solusi yang ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial
XXXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXXX

Step 1 : R
XXXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXXX

Step 2 : U
XXXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXXX

Step 3 : L
XXXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXXX

Step 4 : U
XXXXXXXX
XZ***X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXXX

Step 5 : D
XXXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXXX

Step 6 : R
XXXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXXX

Step 7 : U
XXXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXXX

Step 8 : R
XXXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXXX

>> Waktu eksekusi: 16 ms
>> Banyak iterasi yang dilakukan: 9 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ../test/output/test1.txt
```

A*
(H1)

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
A*
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1

Solusi yang ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX

Step 1 : R
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX

Step 2 : U
XXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Step 3 : L
XXXXXXX
X0***X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Step 4 : U
XXXXXXX
XZ***X
X**X**X
X***OX
X1***LX
X***X*X
XXXXXXX

Step 5 : D
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX

Step 6 : R
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX

Step 7 : U
XXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Step 8 : R
XXXXXXX
X0***X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXXX

>> Waktu eksekusi: 3 ms
>> Banyak iterasi yang dilakukan: 14 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ..\test\output\test1.txt
```

A*
(H2)

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
A*
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H2
```

Solusi yang ditemukan : RULUDRUR
Cost dari Solusi : 87

```
Initial
XXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXX
```

```
Step 1 : R
XXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**Z*X
XXXXXX
```

```
Step 2 : U
XXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXX
```

```
Step 3 : L
XXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXX
```

```
Step 4 : U
XXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXX
```

```
Step 5 : D
XXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXX
```

```
Step 6 : R
XXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**Z*X
XXXXXX
```

```
Step 7 : U
XXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXX
```

```
Step 8 : R
XXXXXX
X0****X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXX
```

```
>> Waktu eksekusi: 17 ms
>> Banyak iterasi yang dilakukan: 14 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ../test/output/test1.txt
```

A*
(H3)

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
A*
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H3
```

Solusi yang ditemukan : RULUDRUR
Cost dari Solusi : 87

```
Initial
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX
```

```
Step 1 : R
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**Z*X*X
XXXXXX
```

```
Step 2 : U
XXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXX
```

```
Step 3 : L
XXXXXX
X0***X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXX
```

```
Step 4 : U
XXXXXX
XZ***X
X**X**X
X***OX
X1***LX
X***X*X
XXXXXX
```

```
Step 5 : D
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX
```

```
Step 6 : R
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**Z*X*X
XXXXXX
```

```
Step 7 : U
XXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXX
```

```
Step 8 : R
XXXXXX
X0***X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXX
```

```
>> Waktu eksekusi: 14 ms
>> Banyak iterasi yang dilakukan: 14 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak Ya
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ../test/output/test1.txt
```


BFS

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
BFS
```

```
Solusi yang ditemukan : RULUDRUR
Cost dari Solusi : 87
```

```
Initial
```

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX
```

```
Step 1 : R
```

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX
```

```
Step 2 : U
```

```
XXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX
```

```
Step 7 : U
```

```
XXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX
```

```
Step 8 : R
```

```
XXXXXXX
X0***X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXXX
```

```
>> Waktu eksekusi: 25 ms
>> Banyak iterasi yang dilakukan: 16 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak)
Ya
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Tidak
```

```
Step 3 : L
```

```
XXXXXXX
X0***X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX
```

```
Step 4 : U
```

```
XXXXXXX
XZ***X
X**X**X
X***OX
X1***LX
X***X*X
XXXXXXX
```

```
Step 5 : D
```

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX
```

```
Step 6 : R
```

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX
```

DFS

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
DFS
```

```
Solusi yang ditemukan : RURLUDRUR
Cost dari Solusi : 104
```

```
Initial
```

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX
```

```
Step 1 : R
```

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX
```

```
Step 2 : U
```

```
XXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX
```

```
Step 7 : R
```

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX
```

```
Step 8 : U
```

```
XXXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX
```

```
Step 9 : R
```

```
XXXXXXX
X0***X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXXX
```

```
>> Waktu eksekusi: 3 ms
>> Banyak iterasi yang dilakukan: 11 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Ya
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Tidak
```

```
Step 3 : R
```

```
XXXXXXX
X0***X
X**X**X
X***ZX
X1***LX
X***X*X
XXXXXXX
```

```
Step 4 : L
```

```
XXXXXXX
X0***X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX
```

```
Step 5 : U
```

```
XXXXXXX
XZ***X
X**X**X
X***OX
X1***LX
X***X*X
XXXXXXX
```

```
Step 6 : D
```

```
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX
```

The image displays two side-by-side screenshots of a software application titled "TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM". The application is designed for pathfinding on a 7x7 grid.

Left Screenshot (UCS):

- The grid shows a path from the start (0) to the goal (Z) using the UCS algorithm. The path is marked with numbers 1, 2, 3, and 4.
- The right panel shows the "Select Algorithm:" dropdown set to "UCS" and the "Select Heuristic:" dropdown set to "H1".
- The "SOLVE" button is visible.
- The "Result Info" section displays: Solution: RULUDRUR, Cost: 87, Iterations: 15, Time: 26 ms, Step: 8/8.
- The "Controls" section shows: [← →] Prev/Next, [ESC] Jump to step.
- The "< Prev" and "Next >" buttons are visible.
- The "SAVE" button is visible at the bottom.

Right Screenshot (GBFS (H1)):

- The grid shows the same path from the start (0) to the goal (Z) using the GBFS (H1) algorithm. The path is marked with numbers 1, 2, 3, and 4.
- The right panel shows the "Select Algorithm:" dropdown set to "GBFS" and the "Select Heuristic:" dropdown set to "H1".
- The "SOLVE" button is visible.
- The "Result Info" section displays: Solution: RULUDRUR, Cost: 87, Iterations: 12, Time: 10 ms, Step: 8/8.
- The "Controls" section shows: [← →] Prev/Next, [ESC] Jump to step.
- The "< Prev" and "Next >" buttons are visible.
- The "SAVE" button is visible at the bottom.

**GBFS
(H1)**

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

oaded: test1.txt

LOAD FILE

Select Algorithm:

GBFS

Select Heuristic:

H1

SOLVE

Result Info

Solution: RULUDRUR

Cost: 87

Iterations: 12

Time: 10 ms

Step: 8/8

Controls:

[← →] Prev/Next
[ESC] Close

[ESC] jump to step

114

SAVE

GBFS (H2)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test1.txt
LOAD FILE

Select Algorithm:
GBFS

Select Heuristic:
H2

SOLVE

Result Info
Solution: RULUDRUR
Cost: 87
Iterations: 12
Time: 20 ms
Step: 8/8

Controls:
[← →] Prev/Next
[ESC] Jump to step

< Prev Next >

SAVE

A 7x7 grid map with obstacles (X) at the top and bottom edges. The start node (0) is at (1,1) and the goal node (Z) is at (4,5). The path RULUDRUR is highlighted in yellow, with nodes 0, 1, and L marked in green.

X	X	X	X	X	X	X
X	0	*	*	*	*	X
X	*	*	X	*	*	X
X	*	*	*	*	Z	X
X	1	*	*	*	L	X
X	*	*	*	X	*	X
X	X	X	X	X	X	X

GBFS (H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test1.txt
LOAD FILE

Select Algorithm:
GBFS

Select Heuristic:
H3

SOLVE

Result Info
Solution: RULUDRUR
Cost: 87
Iterations: 9
Time: 8 ms
Step: 8/8

Controls:
[← →] Prev/Next
[ESC] Jump to step

< Prev Next >

SAVE

A 7x7 grid map identical to the one in the H2 section. The path RULUDRUR is highlighted in yellow, with nodes 0, 1, and L marked in green.

X	X	X	X	X	X	X
X	0	*	*	*	*	X
X	*	*	X	*	*	X
X	*	*	*	*	Z	X
X	1	*	*	*	L	X
X	*	*	*	X	*	X
X	X	X	X	X	X	X

A*
(H1)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test1.txt
LOAD FILE

Select Algorithm:
A*

Select Heuristic:
H1

SOLVE

Result Info
Solution: RULUDRUR
Cost: 87
Iterations: 14
Time: 16 ms
Step: 8/8

Controls:
[← →] Prev/Next
[ESC] Jump to step

< Prev Next >

SAVE

A*
(H2)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test1.txt
LOAD FILE

Select Algorithm:
A*

Select Heuristic:
H2

SOLVE

Result Info
Solution: RULUDRUR
Cost: 87
Iterations: 14
Time: 32 ms
Step: 8/8

Controls:
[← →] Prev/Next
[ESC] Jump to step

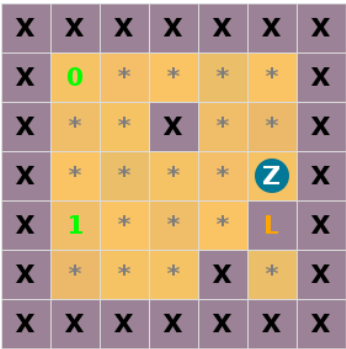
< Prev Next >

SAVE

A*

(H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM



Loaded: test1.txt

LOAD FILE

Select Algorithm:

A*
▼

Select Heuristic:

H3
▼

SOLVE

Result Info

Solution: RULUDRUR

Cost: 87

Iterations: 14

Time: 19 ms

Step: 8/8

< Prev

Next >

SAVE

BFS

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

X	X	X	X	X	X	X
X	0	*	*	*	*	X
X	*	*	X	*	*	X
X	*	*	*	*	Z	X
X	1	*	*	*	L	X
X	*	*	*	X	*	X
X	X	X	X	X	X	X

Loaded: test1.txt

Select Algorithm:

Select Heuristic:

Result Info
 Solution: RULUDRUR
 Cost: 87
 Iterations: 16
 Time: 24 ms
 Step: 8/8

Controls:
 [← →] Prev/Next
 [ESC] Jump to step

DFS

X

X

X

X

X

X

X

X

0

*

*

*

*

X

X

*

*

X

*

*

X

X

*

*

*

*

o

X

X

1

*

*

*

*

X

X

Z

*

*

X

*

X

X

X

X

X

X

X

X

Loaded: test1.txt

LOAD FILE

Select Algorithm:

DFS

Select Heuristic:

H3

SOLVE

Result Info

Solution: RURLUDRUR

Cost: 104

Iterations: 11

Time: 16 ms

Step: 0/9

Controls:

[← →] Prev/Next

[ESC] jump to step

< Prev

Next >

SAVE

4.2 Test Case 2

Papan berbentuk persegi panjang yang valid dengan tembok terdefinisi

INPUT	
<pre>1 6 9 2 XXXXXXXXX 3 X***0***X 4 X*XX*X*0X 5 XZ*X*X*XX 6 X1*****X 7 XXXXXXXXX 8 999 999 999 999 999 999 999 999 9 999 6 7 35 30 25 20 15 999 10 999 5 999 999 15 999 12 1 999 11 999 3 4 999 10 999 9 999 999 12 999 4 3 3 4 5 6 7 999 13 999 999 999 999 999 999 999 999</pre>	
OUTPUT	
	CLI

UCS

```
>> Masukkan file input (EXIT untuk keluar):
test2.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
UCS

Solusi yang ditemukan : URDLDLURD
Cost dari Solusi : 337

Initial
XXXXXXXXXX
X***0***X
X*XX*X*OX
XZ*X*X*XX
X1*****X
XXXXXXXXXX

Step 1 : U
XXXXXXXXXX
XZ**0***X
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 2 : R
XXXXXXXXXX
X***0**ZX
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 3 : D
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 4 : L
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 5 : D
XXXXXXXXXX
X***0***X
X*XX*X*OX
X**X*X*XX
X1****Z*X
XXXXXXXXXX

Step 6 : L
XXXXXXXXXX
X***0***X
X*XX*X*OX
X**X*X*XX
XZ*****X
XXXXXXXXXX

Step 7 : U
XXXXXXXXXX
XZ**0***X
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 8 : R
XXXXXXXXXX
X***0**ZX
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 9 : D
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX

>> Waktu eksekusi: 29 ms
>> Banyak iterasi yang dilakukan: 20 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ../test/output/test2.txt
```


GBFS (H1)

```
>> Masukkan file input (EXIT untuk keluar):
test2.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1
```

```
Solusi yang ditemukan : URDLRLURD
Cost dari Solusi : 350
```

```
Initial
XXXXXXXXXX
X***0***X
X*XX*X*OX
XZ*X*X*XX
X1*****X
XXXXXXXXXX
```

```
Step 1 : U
XXXXXXXXXX
XZ**0***X
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX
```

```
Step 2 : R
XXXXXXXXXX
X***0**ZX
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX
```

```
Step 3 : D
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX
```

```
Step 9 : R
XXXXXXXXXX
X***0**ZX
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX
```

```
Step 10 : D
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX
```

```
>> Waktu eksekusi: 17 ms
>> Banyak iterasi yang dilakukan: 17 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ../test/output/test2.txt
```

```
Step 4 : L
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX
```

```
Step 5 : D
XXXXXXXXXX
X***0***X
X*XX*X*OX
X**X*X*XX
X1*****ZX
XXXXXXXXXX
```

```
Step 6 : R
XXXXXXXXXX
X***0***X
X*XX*X*OX
X**X*X*XX
X1*****ZX
XXXXXXXXXX
```

```
Step 7 : L
XXXXXXXXXX
X***0***X
X*XX*X*OX
X**X*X*XX
XZ*****X
XXXXXXXXXX
```

```
Step 8 : U
XXXXXXXXXX
XZ**0***X
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX
```

GBFS (H2)

```
>> Masukkan file input (EXIT untuk keluar):
test2.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H2

Solusi yang ditemukan : URDLDLURD
Cost dari Solusi : 337

Initial
XXXXXXXXXX
X***0***X
X*XX*X*0X
XZ*X*X*XX
X1*****X
XXXXXXXXXX

Step 1 : U
XXXXXXXXXX
XZ**0***X
X*XX*X*0X
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 2 : R
XXXXXXXXXX
X***0**ZX
X*XX*X*0X
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 3 : D
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 4 : L
XXXXXXXXXX
X***0***X
X*XX*XZ0X
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 5 : D
XXXXXXXXXX
X***0***X
X*XX*X*0X
X**X*X*XX
X1****Z*X
XXXXXXXXXX

Step 6 : L
XXXXXXXXXX
X***0***X
X*XX*X*0X
X**X*X*XX
XZ*****X
XXXXXXXXXX

Step 7 : U
XXXXXXXXXX
XZ**0***X
X*XX*X*0X
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 8 : R
XXXXXXXXXX
X***0**ZX
X*XX*X*0X
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 9 : D
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX

>> Waktu eksekusi: 24 ms
>> Banyak iterasi yang dilakukan: 19 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ../test/output/test2.txt
```

GBFS (H3)

```
>> Masukkan file input (EXIT untuk keluar):  
test2.txt  
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)  
GBFS  
>> Heuristic apa yang anda pilih? (H1/H2/H3)  
H3
```

```
Solusi yang ditemukan : URLDURD  
Cost dari Solusi : 425
```

```
Initial  
XXXXXXXXXX  
X***0***X  
X*XX*X*OX  
XZ*X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 1 : U  
XXXXXXXXXX  
XZ**0***X  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 2 : R  
XXXXXXXXXX  
X***0**ZX  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 3 : L  
XXXXXXXXXX  
XZ**0***X  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 4 : D  
XXXXXXXXXX  
X***0***X  
X*XX*X*OX  
X**X*X*XX  
XZ*****X  
XXXXXXXXXX
```

```
Step 5 : U  
XXXXXXXXXX  
XZ**0***X  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 6 : R  
XXXXXXXXXX  
X***0**ZX  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 7 : D  
XXXXXXXXXX  
X***0***X  
X*XX*X*ZX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
>> Waktu eksekusi: 14 ms  
>> Banyak iterasi yang dilakukan: 12 iterasi  
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :  
Tidak  
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :  
Ya  
>> Disimpan pada ../test/output/test2.txt
```

A*
(H1)

```
>> Masukkan file input (EXIT untuk keluar):  
test2.txt  
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)  
A*  
>> Heuristic apa yang anda pilih? (H1/H2/H3)  
H1
```

```
Solusi yang ditemukan : URDLDLURD  
Cost dari Solusi : 337
```

```
Initial  
XXXXXXXXXX  
X***0***X  
X*XX*X*OX  
XZ*X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 1 : U  
XXXXXXXXXX  
XZ**0***X  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 2 : R  
XXXXXXXXXX  
X***0**ZX  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 3 : D  
XXXXXXXXXX  
X***0***X  
X*XX*X*ZX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 4 : L  
XXXXXXXXXX  
X***0***X  
X*XX*X*ZX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 5 : D  
XXXXXXXXXX  
X***0***X  
X*XX*X*OX  
X**X*X*XX  
X1****7*X  
XXXXXXXXXX
```

```
Step 6 : L  
XXXXXXXXXX  
X***0***X  
X*XX*X*OX  
X**X*X*XX  
XZ*****X  
XXXXXXXXXX
```

```
Step 7 : U  
XXXXXXXXXX  
XZ**0***X  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 8 : R  
XXXXXXXXXX  
X***0**ZX  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 9 : D  
XXXXXXXXXX  
X***0***X  
X*XX*X*ZX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
>> Waktu eksekusi: 19 ms  
>> Banyak iterasi yang dilakukan: 20 iterasi  
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :  
Tidak  
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :  
Ya  
>> Disimpan pada ../test/output/test2.txt
```

A*
(H2)

```
>> Masukkan file input (EXIT untuk keluar):
test2.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
A*
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H2

Solusi yang ditemukan : URDLDLURD
Cost dari Solusi : 337

Initial
XXXXXXXXXX
X***0***X
X*XX*X*OX
XZ*X*X*XX
X1*****X
XXXXXXXXXX

Step 1 : U
XXXXXXXXXX
XZ**0***X
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 2 : R
XXXXXXXXXX
X***0**ZX
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 3 : D
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 4 : L
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 5 : D
XXXXXXXXXX
X***0***X
X*XX*X*OX
X**X*X*XX
X1***Z*X
XXXXXXXXXX

Step 6 : L
XXXXXXXXXX
X***0***X
X*XX*X*OX
X**X*X*XX
XZ*****X
XXXXXXXXXX

Step 7 : U
XXXXXXXXXX
XZ**0***X
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 8 : R
XXXXXXXXXX
X***0**ZX
X*XX*X*OX
X**X*X*XX
X1*****X
XXXXXXXXXX

Step 9 : D
XXXXXXXXXX
X***0***X
X*XX*X*ZX
X**X*X*XX
X1*****X
XXXXXXXXXX

>> Waktu eksekusi: 19 ms
>> Banyak iterasi yang dilakukan: 20 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Tidak
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Disimpan pada ../test/output/test2.txt
```

A*
(H3)

```
>> Masukkan file input (EXIT untuk keluar):  
test2.txt  
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)  
A*  
>> Heuristic apa yang anda pilih? (H1/H2/H3)  
H3
```

```
Solusi yang ditemukan : URDLDLURD  
Cost dari Solusi : 337
```

```
Initial  
XXXXXXXXXX  
X***0***X  
X*XX*X*OX  
XZ*X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 1 : U  
XXXXXXXXXX  
XZ**0***X  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 2 : R  
XXXXXXXXXX  
X***0**ZX  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 3 : D  
XXXXXXXXXX  
X***0***X  
X*XX*X*ZX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 4 : L  
XXXXXXXXXX  
X***0***X  
X*XX*XZOX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 5 : D  
XXXXXXXXXX  
X***0***X  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 6 : L  
XXXXXXXXXX  
X***0***X  
X*XX*X*OX  
X**X*X*XX  
XZ*****X  
XXXXXXXXXX
```

```
Step 7 : U  
XXXXXXXXXX  
XZ**0***X  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 8 : R  
XXXXXXXXXX  
X***0**ZX  
X*XX*X*OX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
Step 9 : D  
XXXXXXXXXX  
X***0***X  
X*XX*X*ZX  
X**X*X*XX  
X1*****X  
XXXXXXXXXX
```

```
>> Waktu eksekusi: 75 ms  
>> Banyak iterasi yang dilakukan: 20 iterasi  
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :  
Tidak  
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :  
Ya  
>> Disimpan pada ../test/output/test2.txt
```

BFS	<pre> >> Masukkan file input (EXIT untuk keluar): test2.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) BFS Solusi yang ditemukan : URLDURD Cost dari Solusi : 425 Initial XXXXXXXXXX X***0***X X*XX*X*0X XZ*X*X*0X X1*****X XXXXXXXXXX Step 1 : U XXXXXXXXXX XZ**0***X X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 2 : R XXXXXXXXXX X***0**ZX X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 3 : L XXXXXXXXXX XZ**0***X X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 4 : D XXXXXXXXXX X***0***X X*XX*X*0X X**X*X*0X XZ*****X XXXXXXXXXX Step 5 : U XXXXXXXXXX XZ**0***X X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 6 : R XXXXXXXXXX X***0**ZX X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 7 : D XXXXXXXXXX X***0***X X*XX*X*ZX X**X*X*0X X1*****X XXXXXXXXXX >> Waktu eksekusi: 21 ms >> Banyak iterasi yang dilakukan: 18 iterasi </pre>
DFS	<pre> >> Masukkan file input (EXIT untuk keluar): test2.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) DFS Solusi yang ditemukan : URLDURD Cost dari Solusi : 425 Initial XXXXXXXXXX X***0***X X*XX*X*0X XZ*X*X*0X X1*****X XXXXXXXXXX Step 1 : U XXXXXXXXXX XZ**0***X X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 2 : R XXXXXXXXXX X***0**ZX X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 3 : L XXXXXXXXXX XZ**0***X X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 4 : D XXXXXXXXXX X***0***X X*XX*X*0X X**X*X*0X XZ*****X XXXXXXXXXX Step 5 : U XXXXXXXXXX XZ**0***X X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 6 : R XXXXXXXXXX X***0**ZX X*XX*X*0X X**X*X*0X X1*****X XXXXXXXXXX Step 7 : D XXXXXXXXXX X***0***X X*XX*X*ZX X**X*X*0X X1*****X XXXXXXXXXX >> Waktu eksekusi: 15 ms >> Banyak iterasi yang dilakukan: 12 iterasi </pre>
GUI	

UCS

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

⏮ ⏪ ⏩ ⏭

X	X	X	X	X	X	X	X	X
X	*	*	*	0	*	*	*	X
X	*	X	X	*	X	*	2	X
X	*	*	X	*	X	*	X	X
X	1	*	*	*	*	*	*	X
X	X	X	X	X	X	X	X	X

Loaded: test2.txt

LOAD FILE

Select Algorithm:

UCS
⌵

Select Heuristic:

H3
⌵

SOLVE

Result Info
 solution: URDLDLURD
 Cost: 337
 Iterations: 20
 Time: 42 ms
 Step: 9/9

Controls:
 [← →] Prev/Next
 [ESC] Jump to step

< Prev

Next >

SAVE

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM
Loaded: test2.txt
LOAD FILE

X	X	X	X	X	X	X	X	X
X	*	*	*	0	*	*	*	X
X	*	X	X	*	X	*	Z	X
X	*	*	X	*	X	*	X	X
X	1	*	*	*	*	*	*	X
X	X	X	X	X	X	X	X	X

Select Algorithm:
GBFS

Select Heuristic:
H1

SOLVE

Result Info

Solution: URDLRLURD
Cost: 350
Iterations: 17
Time: 34 ms
Step: 10/10

Controls:
[← →] Prev/Next
[ESC] Jump to step

< Prev Next >

SAVE

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

X	X	X	X	X	X	X	X	X	X
X	*	*	*	S	*	*	*	*	X
X	*	X	X	*	X	*	Z	*	X
X	*	*	X	*	X	*	X	X	X
X	1	*	*	*	*	*	*	*	X
X	X	X	X	X	X	X	X	X	X

Loaded: test2.txt
LOAD FILE

Select Algorithm:

Select Heuristic:

SOLVE

Result Info

 Solution: URDLDLURD
 Cost: 337
 Iterations: 19
 Time: 38 ms
 Steps: 9/9

Controls:
 [← →] Prev/Next
 [ESC] Jump to step

< Prev
Next >

SAVE

GBFS (H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

X	X	X	X	X	X	X	X	X
X	*	*	*	0	*	*	*	X
X	*	X	X	*	X	*	Z	X
X	*	*	X	*	X	*	X	X
X	1	*	*	*	*	*	*	X
X	X	X	X	X	X	X	X	X

Loaded: test2.txt

Select Algorithm:

GBFS
▾

Select Heuristic:

H3
▾

Result Info
 Solution: URLDURD
 Cost: 425
 Iterations: 12
 Time: 23 ms
 Step: 7/7

Controls:
 [← →] Prev/Next
 [ESC] Jump to step

A*

(H1)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

X	X	X	X	X	X	X	X	X
X	*	*	*	0	*	*	*	X
X	*	X	X	*	X	*	Z	X
X	*	*	X	*	X	*	X	X
X	1	*	*	*	*	*	*	X
X	X	X	X	X	X	X	X	X

Loaded: test2.txt

Select Algorithm:

Select Heuristic:

Result Info
 Solution: URDLDLURD
 Cost: 337
 Iterations: 20
 Time: 38 ms
 Step: 9/9

Controls:
 [← →] Prev/Next
 [ESC] Jump to step

A*

(H2)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

X	X	X	X	X	X	X	X	X
X	*	*	*	0	*	*	*	X
X	*	X	X	*	X	Z	*	X
X	*	*	X	*	X	*	X	X
X	1	*	*	*	*	*	*	X
X	X	X	X	X	X	X	X	X

Loaded: test2.txt

Select Algorithm:

Select Heuristic:

Result Info
 Solution: URDLDLURD
 Cost: 337
 Iterations: 20
 Time: 32 ms
 Step: 9/9

Controls:
[← →] Prev/Next
[ESC] Jump to step

A*
(H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test2.txt
LOAD FILE

Select Algorithm:
A*

Select Heuristic:
H3

SOLVE

Result Info
Solution: URDLDLURD
Cost: 337
Iterations: 20
Time: 35 ms
Step: 9/9

Controls:
[← →] Prev/Next
[ESC] Jump to step

< Prev Next >

SAVE

BFS

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test2.txt
LOAD FILE

Select Algorithm:
BFS

Select Heuristic:
H3

SOLVE

Result Info
Solution: URLDLURD
Cost: 425
Iterations: 18
Time: 32 ms
Step: 0/7

Controls:
[← →] Prev/Next
[ESC] Jump to step

< Prev Next >

SAVE

DFS

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test2.txt

LOAD FILE

Select Algorithm:

DFS

Select Heuristic:

H3

SOLVE

Result Info

Solution: URLDURD

Cost: 425

Iterations: 18

Time: 32 ms

Step: 0/7

Controls:

[← →] Prev/Next

[ESC] Jump to step

< Prev

Next >

SAVE

X X X X X X X X X

X * * * 0 * * * X

X * X X * X * O X

X Z * X * X * X X

X 1 * * * * * * X

X X X X X X X X X

4.3 Test Case 3

Papan tidak memiliki tembok yang dapat memberhentikan *user* ketika *sliding* sehingga tidak ada solusi.

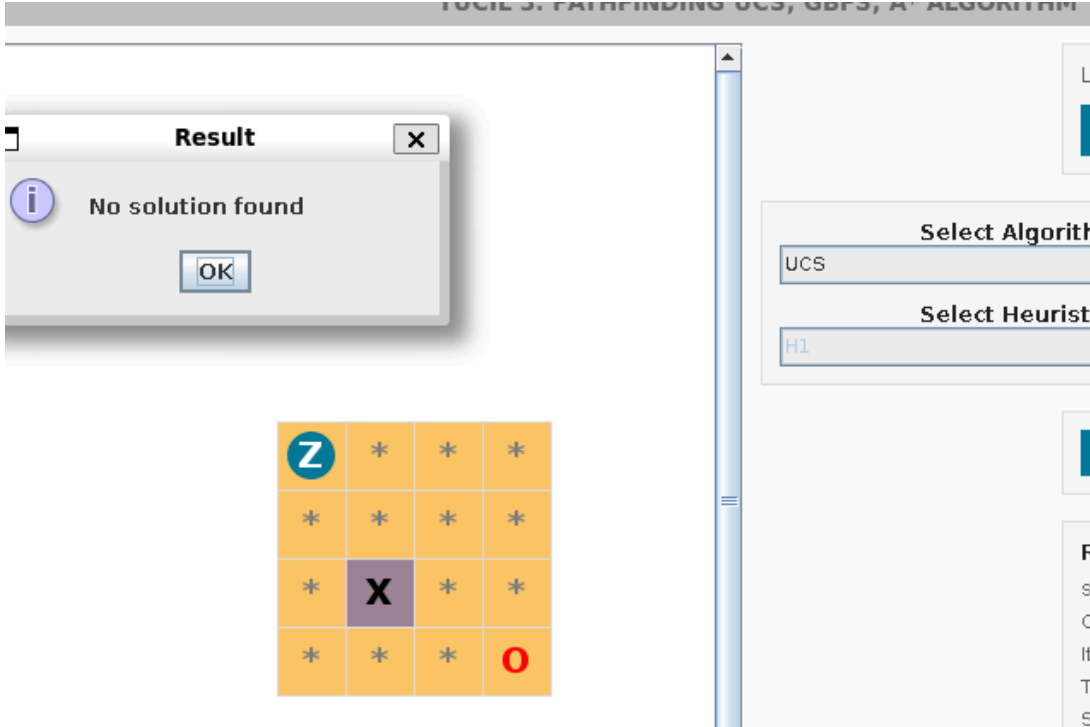
INPUT	
	1 4 4 2 Z*** 3 **** 4 *X** 5 ***0 6 1 1 1 1 7 1 1 1 1 8 1 999 1 1 9 1 1 1 1
OUTPUT	
	CLI

UCS	<pre> test3.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) UCS Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt 1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! </pre>
GBFS (H1)	<pre> >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) GBFS >> Heuristic apa yang anda pilih? (H1/H2/H3) H1 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt 1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! </pre>
GBFS (H2)	<pre> >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) GBFS >> Heuristic apa yang anda pilih? (H1/H2/H3) H2 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt </pre>

	<pre> 1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! 12 </pre>
GBFS (H3)	<pre> >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) GBFS >> Heuristic apa yang anda pilih? (H1/H2/H3) H3 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt </pre> <pre> 1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! 12 </pre>
A* (H1)	<pre> >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) A* >> Heuristic apa yang anda pilih? (H1/H2/H3) H1 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt </pre>

	<pre> 1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! </pre>
A* (H2)	<pre> >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) A* >> Heuristic apa yang anda pilih? (H1/H2/H3) H2 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt </pre> <pre> 1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! </pre>

A* (H3)	<pre>>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) A* >> Heuristic apa yang anda pilih? (H1/H2/H3) H3 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt</pre> <pre>1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! 12</pre>
BFS	<pre>>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) BFS Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt</pre> <pre>1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! 12</pre>

DFS	<pre> >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) DFS Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test3.txt 1 === Iteration 1 === 2 Initial 3 Z*** 4 **** 5 *X** 6 ***0 7 8 -> U GAME OVER (Keluar dari batas papan)! 9 -> D GAME OVER (Keluar dari batas papan)! 10 -> L GAME OVER (Keluar dari batas papan)! 11 -> R GAME OVER (Keluar dari batas papan)! </pre>
	GUI
UCS	

GBFS (H1)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test3.txt
LOAD FILE

Select Algorithm:
GBFS

Select Heuristic:
H1

SOLVE

Result Info
Solution: -
Cost: -
Iterations: -
Time: -
Step: 0/?

Controls:
[← →] Prev/Next
[ESC] Jump to step
< Prev **Next >**

SAVE

Result
No solution found
OK

Z	*	*	*
*	*	*	*
*	X	*	*
*	*	*	O

GBFS (H2)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test3.txt
LOAD FILE

Select Algorithm:
GBFS

Select Heuristic:
H2

SOLVE

Result Info
Solution: -
Cost: -
Iterations: -
Time: -
Step: 0/?

Controls:

Result
No solution found
OK

Z	*	*	*
*	*	*	*
*	X	*	*
*	*	*	O

**GBFS
(H3)**

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test3.txt
LOAD FILE

Select Algorithm:
GBFS

Select Heuristic:
H3

SOLVE

Result
No solution found
OK

Z	*	*	*
*	*	*	*
*	X	*	*
*	*	*	O

Result Info
Solution: -
Cost: -
Iterations: -
Time: -
Step: 0/?

**A*
(H1)**

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test3.txt
LOAD FILE

Select Algorithm:
A*

Select Heuristic:
H1

SOLVE

Result
No solution found
OK

Z	*	*	*
*	*	*	*
*	X	*	*
*	*	*	O

Result Info
Solution: -
Cost: -
Iterations: -
Time: -
Step: 0/?

A*
(H2)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Result

No solution found

OK

Z	*	*	*
*	*	*	*
*	X	*	*
*	*	*	O

Loaded: test3.txt

LOAD FILE

Select Algorithm:

A*

Select Heuristic:

H2

SOLVE

Result Info

Solution: -

Cost: -

Iterations: -

Time: -

Step: 0/?

A*
(H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Result

No solution found

OK

Z	*	*	*
*	*	*	*
*	X	*	*
*	*	*	O

Loaded: test3.txt

LOAD FILE

Select Algorithm:

A*

Select Heuristic:

H3

SOLVE

Result Info

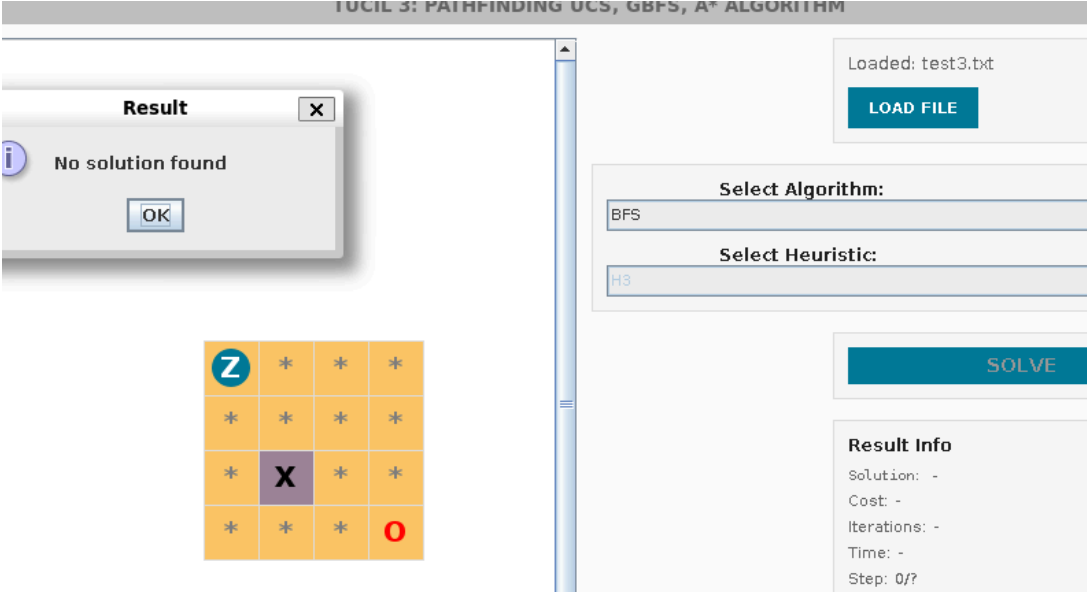
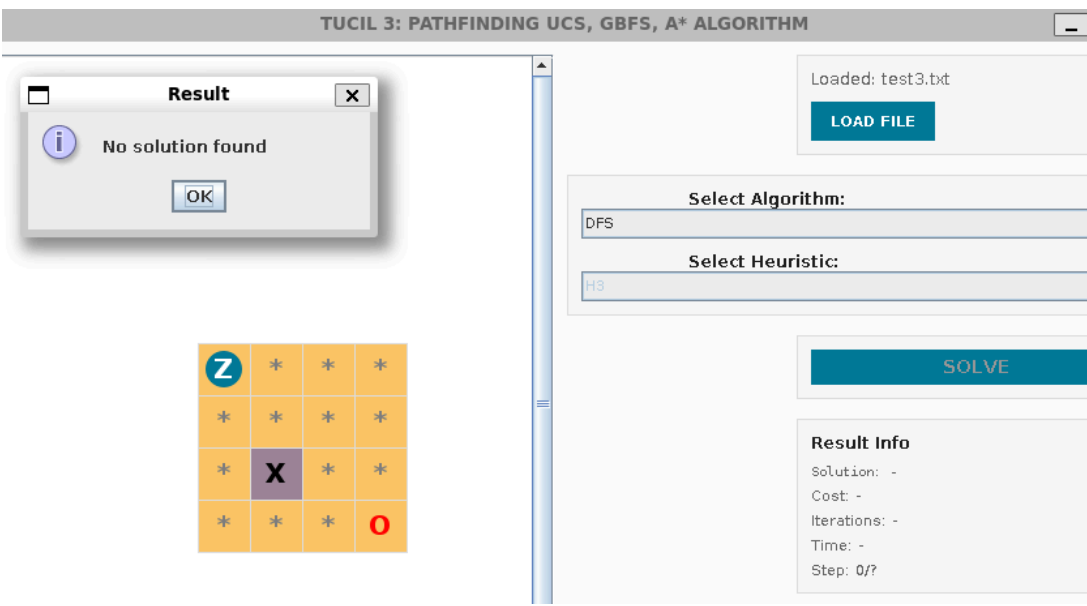
Solution: -

Cost: -

Iterations: -

Time: -

Step: 0/?

BFS	
DFS	

4.4 Test Case 4

Papan tidak valid, ukurannya tidak sesuai dengan input N M awal.

INPUT

<pre> 1 4 4 2 XXXX 3 XZOX 4 XXXX 5 999 999 999 999 6 999 1 1 999 7 999 999 999 999 </pre>	
OUTPUT	
<pre> >> Masukkan file input (EXIT untuk keluar): test4.txt [ERROR] Panjang baris ke-4 tidak sama dengan M! Ngitung dong atleast :(</pre>	

4.5 Test Case 5

Papan valid, tetapi *GOAL* dikelilingi dengan lava, sehingga *user* tidak mempunyai cara untuk mencapainya.

INPUT	
<pre> 1 5 5 2 XXXXX 3 X**LX 4 X*LOX 5 XZ*LX 6 XXXXX 7 999 999 999 999 999 8 999 7 35 999 999 9 999 5 999 15 999 10 999 3 9 999 999 11 999 999 999 999 999 999 999 999 999 </pre>	
OUTPUT	
	CLI
UCS	<pre> >> Masukkan file input (EXIT untuk keluar): test5.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) UCS Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test5.txt </pre>

	<pre> 1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*L0X 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*L0X 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*L0X 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)! 27 28 </pre>
GBFS (H1)	<pre> 1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*L0X 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*L0X 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*L0X 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)! 27 28 </pre>
GBFS (H2)	<pre> >> Masukkan file input (EXIT untuk keluar): test5.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) GBFS >> Heuristic apa yang anda pilih? (H1/H2/H3) H2 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test5.txt </pre>

	<pre> 1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*LOX 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*LOX 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*LOX 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)! 27 </pre>
GBFS (H3)	<pre> >> Masukkan file input (EXIT untuk keluar): test5.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) GBFS >> Heuristic apa yang anda pilih? (H1/H2/H3) H3 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test5.txt </pre> <pre> 1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*LOX 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*LOX 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*LOX 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)! 27 </pre>

A* (H1)	<pre>>> Masukkan file input (EXIT untuk keluar): test5.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) A* >> Heuristic apa yang anda pilih? (H1/H2/H3) H1 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test5.txt</pre> <pre>1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*LOX 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*LOX 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*LOX 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)!</pre>
A* (H2)	<pre>>> Masukkan file input (EXIT untuk keluar): test5.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) A* >> Heuristic apa yang anda pilih? (H1/H2/H3) H2 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test5.txt</pre>

	<pre> 1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*LOX 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*LOX 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*LOX 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)! </pre>
A* (H3)	<pre> >> Masukkan file input (EXIT untuk keluar): test5.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) A* >> Heuristic apa yang anda pilih? (H1/H2/H3) H3 Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test5.txt </pre> <pre> 1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*LOX 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*LOX 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*LOX 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)! </pre>

BFS	<pre>>> Masukkan file input (EXIT untuk keluar): test5.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) BFS Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test5.txt</pre> <pre>1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*LOX 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*LOX 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*LOX 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)!</pre>
DFS	<pre>>> Masukkan file input (EXIT untuk keluar): test5.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) DFS Solusi tidak ditemukan! Anda bisa melihat iterasi yang dilakukan di file test/iteration/test5.txt</pre> <pre>1 === Iteration 1 === 2 Initial 3 XXXXX 4 X**LX 5 X*LOX 6 XZ*LX 7 XXXXX 8 9 -> R GAME OVER (Melewati Lava)! 10 11 === Iteration 2 === 12 Initial 13 XXXXX 14 X**LX 15 X*LOX 16 XZ*LX 17 XXXXX 18 19 Step 1 : U 20 XXXXX 21 XZ*LX 22 X*LOX 23 X**LX 24 XXXXX 25 26 -> R GAME OVER (Melewati Lava)!</pre>

	GUI																									
UCS	TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM Result No solution found OK <table><tr><td>X</td><td>X</td><td>X</td><td>X</td><td>X</td></tr><tr><td>X</td><td>*</td><td>*</td><td>L</td><td>X</td></tr><tr><td>X</td><td>*</td><td>L</td><td>O</td><td>X</td></tr><tr><td>X</td><td>Z</td><td>*</td><td>L</td><td>X</td></tr><tr><td>X</td><td>X</td><td>X</td><td>X</td><td>X</td></tr></table> Loaded: test5.txt LOAD FILE Select Algorithm: UCS Select Heuristic: H3 SOLVE Result Info Solution: - Cost: - Iterations: - Time: - Step: 0/?	X	X	X	X	X	X	*	*	L	X	X	*	L	O	X	X	Z	*	L	X	X	X	X	X	X
X	X	X	X	X																						
X	*	*	L	X																						
X	*	L	O	X																						
X	Z	*	L	X																						
X	X	X	X	X																						
GBFS (H1)	TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM Result No solution found OK <table><tr><td>X</td><td>X</td><td>X</td><td>X</td><td>X</td></tr><tr><td>X</td><td>*</td><td>*</td><td>L</td><td>X</td></tr><tr><td>X</td><td>*</td><td>L</td><td>O</td><td>X</td></tr><tr><td>X</td><td>Z</td><td>*</td><td>L</td><td>X</td></tr><tr><td>X</td><td>X</td><td>X</td><td>X</td><td>X</td></tr></table> Loaded: test5.txt LOAD FILE Select Algorithm: GBFS Select Heuristic: H1 SOLVE Result Info Solution: - Cost: - Iterations: - Time: - Step: 0/?	X	X	X	X	X	X	*	*	L	X	X	*	L	O	X	X	Z	*	L	X	X	X	X	X	X
X	X	X	X	X																						
X	*	*	L	X																						
X	*	L	O	X																						
X	Z	*	L	X																						
X	X	X	X	X																						

GBFS
(H2)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test5.txt
LOAD FILE

Select Algorithm:
GBFS

Select Heuristic:
H2

Result
No solution found
OK

X	X	X	X	X
X	*	*	L	X
X	*	L	O	X
X	Z	*	L	X
X	X	X	X	X

Result Info
Solution: -
Cost: -
Iterations: -
Time: -
Step: 0/?

GBFS
(H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test5.txt
LOAD FILE

Select Algorithm:
GBFS

Select Heuristic:
H3

Result
No solution found
OK

X	X	X	X	X
X	*	*	L	X
X	*	L	O	X
X	Z	*	L	X
X	X	X	X	X

Result Info
Solution: -
Cost: -
Iterations: -
Time: -
Step: 0/?

A*
(H1)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Result

No solution found

OK

X	X	X	X	X
X	*	*	L	X
X	*	L	O	X
X	Z	*	L	X
X	X	X	X	X

Select Algorithm:
A*

Select Heuristic:
H1

Loaded: tes

LOAD FI

Result Info

Solution: -

Cost: -

Iterations: -

Time: -

Step: 0/?

A*
(H2)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Result

No solution found

OK

X	X	X	X	X
X	*	*	L	X
X	*	L	O	X
X	Z	*	L	X
X	X	X	X	X

Select Algorithm:
A*

Select Heuristic:
H2

Loaded: tes

LOAD FI

Result Info

Solution: -

Cost: -

Iterations: -

Time: -

Step: 0/?

A*
(H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Result

No solution found

OK

X	X	X	X	X
X	*	*	L	X
X	*	L	O	X
X	Z	*	L	X
X	X	X	X	X

Select Algorithm:
A*

Select Heuristic:
H3

Load

Result

Solut.

Cost:

Itera:

Time:

Step:

BFS

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Result

No solution found

OK

X	X	X	X	X
X	*	*	L	X
X	*	L	O	X
X	Z	*	L	X
X	X	X	X	X

Select Algorithm:
BFS

Select Heuristic:
H3

Load

Result

Solut.

Cost:

Itera:

Time:

Step:

DFS

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Result

No solution found

OK

X	X	X	X	X
X	*	*	L	X
X	*	L	O	X
X	Z	*	L	X
X	X	X	X	X

Loaded:

LOAD

Select Algorithm:

DFS

Select Heuristic:

H3

Result

Solution

Cost: -

Iterations

Time: -

Step: 0/?

4.6 Test Case 6

INPUT

```

8 10
XXXXXXXXXX
XZ**0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
999 999 999 999 999 999 999 999 999 999
999 2 36 34 22 999 40 44 30 999
999 3 999 999 5 999 7 999 9 999
999 4 999 6 7 8 9 6 10 999
999 3 999 7 8 999 5 4 3 999
999 2 2 3 999 999 4 5 2 999
999 1 1 1 2 2 999 6 6 999
999 999 999 999 999 999 999 999 999 999

```


OUTPUT		
	CLI	
UCS	<pre>>> Masukkan file input (EXIT untuk keluar): test6.txt >> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS) UCS Solusi yang ditemukan : RDLURULDR Cost dari Solusi : 320 Initial XXXXXXXXXX XZ**0X***X X*XX*X*X*X X*X***1*X X*X**X***X X***XX2*0X X*****X*X XXXXXXXXXX Step 1 : R XXXXXXXXXX X***ZX***X X*XX*X*X*X X*X***1*X X*X**X***X X***XX2*0X X*****X*X XXXXXXXXXX Step 2 : D XXXXXXXXXX X***0X***X X*XX*X*X*X X*X***1*X X*X*ZX***X X***XX2*0X X*****X*X XXXXXXXXXX</pre>	<pre>Step 3 : L XXXXXXXXXX X***0X***X X*XX*X*X*X X*X***1*X X*XZ*X***X X***XX2*0X X*****X*X XXXXXXXXXX Step 4 : U XXXXXXXXXX X***0X***X X*XX*X*X*X X*XZ***1*X X*X**X***X X***XX2*0X X*****X*X XXXXXXXXXX Step 5 : R XXXXXXXXXX X***0X***X X*XX*X*X*X X*X***1ZX X*X**X***X X***XX2*0X X*****X*X XXXXXXXXXX Step 6 : U XXXXXXXXXX X***0X**ZX X*XX*X*X*X X*X***1*X X*X**X***X X***XX2*0X X*****X*X XXXXXXXXXX</pre>

```
Step 7 : L
XXXXXXXXXX
X***0XZ**X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 8 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XXZ*0X
X*****X**X
XXXXXXXXXX

Step 9 : R
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*ZX
X*****X**X
XXXXXXXXXX

>> Waktu eksekusi: 62 ms
>> Banyak iterasi yang dilakukan: 31 iterasi
```

GBFS (H1)

```
>> Masukkan file input (EXIT untuk keluar):
test6.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1
```

```
Solusi yang ditemukan : RDLURULDR
Cost dari Solusi : 320
```

```
Initial
XXXXXXXXXX
XZ**0X***X
X*XX(*X*X*X
X*X****1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 1 : R
XXXXXXXXXX
X***ZX***X
X*XX(*X*X*X
X*X****1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 2 : D
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X****1*X
X*X*ZX***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 7 : L
XXXXXXXXXX
X***0XZ***X
X*XX(*X*X*X
X*X****1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 8 : D
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X****1*X
X*X**X***X
X***XXZ*0X
X*****X**X
XXXXXXXXXX
```

```
Step 9 : R
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X****1*X
X*X**X***X
X***XX2*ZX
X*****X**X
XXXXXXXXXX
```

```
>> Waktu eksekusi: 66 ms
>> Banyak iterasi yang dilakukan: 19 iterasi
```

```
Step 3 : L
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X****1*X
X*XZ*X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 4 : U
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*XZ***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 5 : R
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X****1ZX
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 6 : U
XXXXXXXXXX
X***0X**ZX
X*XX(*X*X*X
X*X****1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

GBFS (H2)

```
>> Masukkan file input (EXI1 untuk keluar):
test6.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H2

Solusi yang ditemukan : RDLURDULDR
Cost dari Solusi : 346

Initial
XXXXXXXXXX
XZ**0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 1 : R
XXXXXXXXXX
X***ZX***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 2 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X*ZX***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 3 : L
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*XZ*X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 4 : U
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*XZ***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 5 : R
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1ZX
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 6 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****XZX
XXXXXXXXXX

Step 7 : U
XXXXXXXXXX
X***0X**ZX
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 8 : L
XXXXXXXXXX
X***0XZ***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX

Step 9 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XXZ*0X
X*****X**X
XXXXXXXXXX

Step 10 : R
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2ZX
X*****X**X
XXXXXXXXXX

>> Waktu eksekusi: 35 ms
>> Banyak iterasi yang dilakukan: 18 iterasi
```

GBFS (H3)

```
>> Masukkan file input (EXIT untuk keluar):  
test6.txt  
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)  
GBFS H3
```

```
Solusi yang ditemukan : RDLURULDR  
Cost dari Solusi : 320
```

```
Initial  
XXXXXXXXXX  
XZ**0X***X  
X*XX(*X*X*X  
X*X****1*X  
X*X***X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 1 : R  
XXXXXXXXXX  
X***ZX***X  
X*XX(*X*X*X  
X*X****1*X  
X*X***X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 2 : D  
XXXXXXXXXX  
X***0X***X  
X*XX(*X*X*X  
X*X****1*X  
X*X*ZX***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 3 : L  
XXXXXXXXXX  
X***0X***X  
X*XX(*X*X*X  
X*X****1*X  
X*XZ*X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 8 : D  
XXXXXXXXXX  
X***0X***X  
X*XX(*X*X*X  
X*X****1*X  
X*X***X***X  
X***XXZ*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 9 : R  
XXXXXXXXXX  
X***0X***X  
X*XX(*X*X*X  
X*X****1*X  
X*X***X***X  
X***XX2*ZX  
X*****X**X  
XXXXXXXXXX
```

```
>> Waktu eksekusi: 50 ms  
>> Banyak iterasi yang dilakukan: 30 iterasi
```

```
Step 4 : U  
XXXXXXXXXX  
X***0X***X  
X*XX(*X*X*X  
X*XZ***1*X  
X*X***X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 5 : R  
XXXXXXXXXX  
X***0X***X  
X*XX(*X*X*X  
X*X****1ZX  
X*X***X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 6 : U  
XXXXXXXXXX  
X***0X**ZX  
X*XX(*X*X*X  
X*X****1*X  
X*X***X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 7 : L  
XXXXXXXXXX  
X***0XZ**X  
X*XX(*X*X*X  
X*X****1*X  
X*X***X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

A*
(H1)

```
>> Masukkan file input (EX11 untuk keluar):
test6.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
A*
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1
```

```
Solusi yang ditemukan : RDLURULDR
Cost dari Solusi : 320
```

```
Initial
XXXXXXXXXX
XZ**0X***X
X*XX(*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 1 : R
XXXXXXXXXX
X***ZX***X
X*XX(*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 2 : D
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X***1*X
X*X*ZX***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 7 : L
XXXXXXXXXX
X***0XZ**X
X*XX(*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 8 : D
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X***1*X
X*X**X***X
X***XXZ*0X
X*****X**X
XXXXXXXXXX
```

```
Step 9 : R
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X***1*X
X*X**X***X
X***XX2*ZX
X*****X**X
XXXXXXXXXX
```

```
>> Waktu eksekusi: 64 ms
>> Banyak iterasi yang dilakukan: 30 iterasi
```

```
Step 3 : L
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X***1*X
X*XZ*X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 4 : U
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*XZ***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 5 : R
XXXXXXXXXX
X***0X***X
X*XX(*X*X*X
X*X***1ZX
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 6 : U
XXXXXXXXXX
X***0X**ZX
X*XX(*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

A*
(H2)

```
>> Masukkan file input (EXIT untuk keluar):  
test6.txt  
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)  
A*  
>> Heuristic apa yang anda pilih? (H1/H2/H3)  
H2
```

```
Solusi yang ditemukan : RDLURULDR  
Cost dari Solusi : 320
```

```
Initial  
XXXXXXXXXX  
XZ**0X***X  
X*XX*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 1 : R  
XXXXXXXXXX  
X***ZX***X  
X*XX*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 2 : D  
XXXXXXXXXX  
X***0X***X  
X*XX*X*X*X  
X*X***1*X  
X*XZX***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 7 : L  
XXXXXXXXXX  
X***0XZ**X  
X*XX*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 8 : D  
XXXXXXXXXX  
X***0X***X  
X*XX*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 9 : R  
XXXXXXXXXX  
X***0X***X  
X*XX*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*ZX  
X*****X**X  
XXXXXXXXXX
```

```
>> Waktu eksekusi: 33 ms  
>> Banyak iterasi yang dilakukan: 30 iterasi
```

```
Step 3 : L  
XXXXXXXXXX  
X***0X***X  
X*XX*X*X*X  
X*X***1*X  
X*XZ*X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 4 : U  
XXXXXXXXXX  
X***0X***X  
X*XX*X*X*X  
X*XZ***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 5 : R  
XXXXXXXXXX  
X***0X***X  
X*XX*X*X*X  
X*X***1ZX  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 6 : U  
XXXXXXXXXX  
X***0X**ZX  
X*XX*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

A*
(H3)

```
>> Masukkan file input (EXIT untuk keluar):  
test6.txt  
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)  
A*  
>> Heuristic apa yang anda pilih? (H1/H2/H3)  
H3
```

```
Solusi yang ditemukan : RDLURULDR  
Cost dari Solusi : 320
```

```
Initial  
XXXXXXXXXX  
XZ**0X***X  
X*XX*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 1 : R  
XXXXXXXXXX  
X***ZX***X  
X*XX*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 2 : D  
XXXXXXXXXX  
X***0X***X  
X*XX*X*X*X  
X*X***1*X  
X*X*ZX***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 7 : L  
XXXXXXXXXX  
X***0XZ**X  
X*XX(X*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 8 : D  
XXXXXXXXXX  
X***0X***X  
X*XX(X*X*X*X  
X*X***1*X  
X*X**X***X  
X***XXZ*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 9 : R  
XXXXXXXXXX  
X***0X***X  
X*XX(X*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*ZX  
X*****X**X  
XXXXXXXXXX
```

```
>> Waktu eksekusi: 44 ms  
>> Banyak iterasi yang dilakukan: 30 iterasi
```

```
Step 3 : L  
XXXXXXXXXX  
X***0X***X  
X*XX(X*X*X*X  
X*X***1*X  
X*XZ*X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 4 : U  
XXXXXXXXXX  
X***0X***X  
X*XX(X*X*X*X  
X*XZ***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 5 : R  
XXXXXXXXXX  
X***0X***X  
X*XX(X*X*X*X  
X*X***1ZX  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```

```
Step 6 : U  
XXXXXXXXXX  
X***0X**ZX  
X*XX(X*X*X*X  
X*X***1*X  
X*X**X***X  
X***XX2*0X  
X*****X**X  
XXXXXXXXXX
```


BFS

```
>> Masukkan file input (EXIT untuk keluar):
test6.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
BFS
```

```
Solusi yang ditemukan : RDLURULDR
Cost dari Solusi : 320
```

```
Initial
XXXXXXXXXX
XZ**0X***X
X*XX*X*X*X
X*X****1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 1 : R
XXXXXXXXXX
X***ZX***X
X*XX*X*X*X
X*X****1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 2 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X****1*X
X*X+ZX***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 7 : L
XXXXXXXXXX
X***0XZ**X
X*XX*X*X*X
X*X****1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 8 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X****1*X
X*X**X***X
X***XXZ*0X
X*****X**X
XXXXXXXXXX
```

```
Step 9 : R
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X****1*X
X*X**X***X
X***XX2*ZX
X*****X**X
XXXXXXXXXX
```

```
>> Waktu eksekusi: 43 ms
>> Banyak iterasi yang dilakukan: 27 iterasi
```

```
Step 3 : L
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X****1*X
X*XZ*X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 4 : U
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*XZ***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 5 : R
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X****1ZX
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 6 : U
XXXXXXXXXX
X***0X**ZX
X*XX*X*X*X
X*X****1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

DFS

```
>> Masukkan file input (EXIT untuk keluar):
test6.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS/DFS)
DFS
```

```
Solusi yang ditemukan : RDLRDULDR
Cost dari Solusi : 360
```

```
Initial
XXXXXXXXXX
XZ**0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 1 : R
XXXXXXXXXX
X***ZX***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 2 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X*ZX***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 7 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X*ZX
XXXXXXXXXX
```

```
Step 8 : U
XXXXXXXXXX
X***0X**ZX
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 9 : L
XXXXXXXXXX
X***0XZ**X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 10 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XXZ*0X
X*****X**X
XXXXXXXXXX
```

```
Step 11 : R
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*ZX
X*****X**X
XXXXXXXXXX
```

```
>> Waktu eksekusi: 88 ms
>> Banyak iterasi yang dilakukan: 26 iterasi
```

```
Step 3 : L
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*XZ*X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 4 : D
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1*X
X*X**X***X
X***XX2*0X
X**ZX**X**X
XXXXXXXXXX
```

```
Step 5 : U
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*XZ***1*X
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

```
Step 6 : R
XXXXXXXXXX
X***0X***X
X*XX*X*X*X
X*X***1ZX
X*X**X***X
X***XX2*0X
X*****X**X
XXXXXXXXXX
```

	GUI
UCS	TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM X X X X X X X X X * * * 0 X * * * X * X X * X * X * X * X * * * * 1 * X * X * * X * * * X * * * X X 2 * Z X * * * * * X * * X X X X X X X X X X Loaded: test6.txt LOAD FILE Select Algorithm: UCS Select Heuristic: H3 SOLVE Result Info Solution: RDLURULDR Cost: 320 Iterations: 31 Time: 102 ms Step: 9/9 Controls: [← →] Prev/Next [ESC] Jump to step < Prev Next >
GBFS (H1)	TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM X X X X X X X X X * * * 0 X * * * X * X X * X * X * X * X * * * * 1 * X * X * * X * * * X * * * X X 2 * Z X * * * * * X * * X X X X X X X X X X Loaded: test6.txt LOAD FILE Select Algorithm: GBFS Select Heuristic: H1 SOLVE Result Info Solution: RDLURULDR Cost: 320 Iterations: 19 Time: 27 ms Step: 9/9 Controls: [← →] Prev/Next [ESC] Jump to step < Prev Next > SAVE

GBFS (H2)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

X	X	X	X	X	X	X	X	X
Z	*	*	0	X	*	*	*	X
*	X	X	*	X	*	X	*	X
*	X	*	*	*	*	1	*	X
*	X	*	*	X	*	*	*	X
*	*	*	X	X	2	*	O	X
*	*	*	*	*	X	*	*	X
X	X	X	X	X	X	X	X	X

Loaded: test6.txt

LOAD FILE

Select Algorithm:

GBFS

Select Heuristic:

H2

SOLVE

Result Info

Solution: RDLURDULDR

Cost: 346

Iterations: 18

Time: 22 ms

Step: 0/10

Controls:
[← →] Prev/Next
[ESC] jump to step

< Prev
Next >

SAVE

GBFS (H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test6.txt

LOAD FILE

Select Algorithm:

GBFS
▼

Select Heuristic:

H3
▼

SOLVE

Result Info

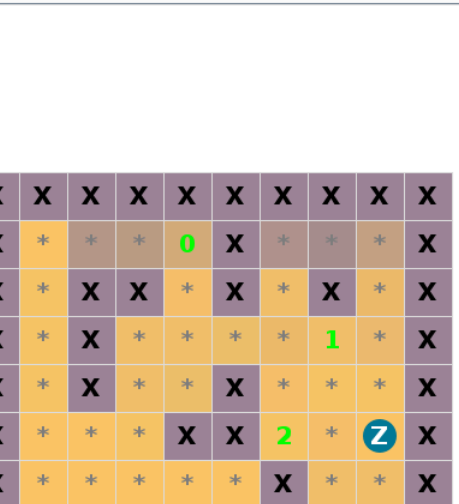
Solution: RDLURULDR
Cost: 320
Iterations: 19
Time: 229 ms
Step: 9/9

Controls:
[← →] Prev/Next
[F5] Jump to step

A*

(H1)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM



Loaded: test6.txt

Select Algorithm:

Select Heuristic:

Result Info
Solution: RDLURULDR
Cost: 320
Iterations: 30
Time: 44 ms
Step: 9/9

Controls:
[← →] Prev/Next
[ESC] jump to step

A* (H2)

A*
(H3)

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test6.txt
LOAD FILE

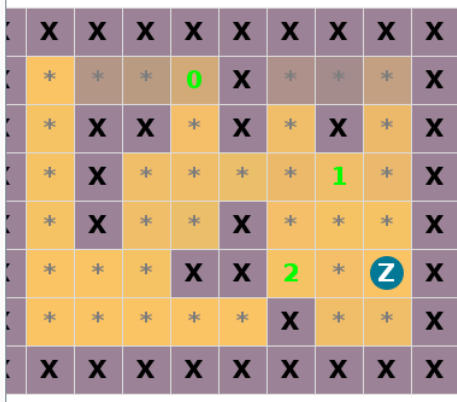
Select Algorithm:
A*

Select Heuristic:
H3

SOLVE

Result Info
Solution: RDLURULDR
Cost: 320
Iterations: 30
Time: 56 ms
Step: 9/9

Controls:
[← →] Prev/Next
[ESC] Turn to step



BFS

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

Loaded: test6.txt
LOAD FILE

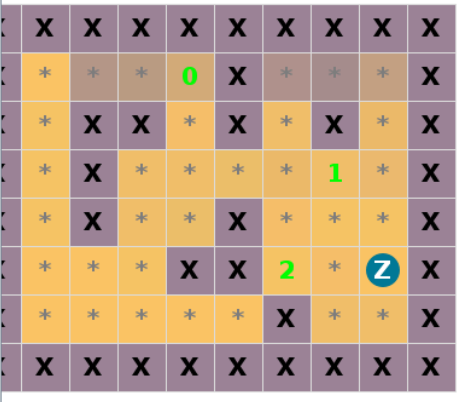
Select Algorithm:
BFS

Select Heuristic:
H3

SOLVE

Result Info
Solution: RDLURULDR
Cost: 320
Iterations: 27
Time: 20 ms
Step: 9/9

Controls:
[← →] Prev/Next
[ESC] Turn to step



DFS

TUCIL 3: PATHFINDING UCS, GBFS, A* ALGORITHM

X	X	X	X	X	X	X	X	X
*	*	*	0	X	*	*	*	X
*	X	X	*	X	*	X	*	X
*	X	*	*	*	*	1	*	X
*	X	*	*	X	*	*	*	X
*	*	*	X	X	2	*	Z	X
*	*	*	*	*	X	*	*	X
X	X	X	X	X	X	X	X	X

Loaded: test6.txt

LOAD FILE

Select Algorithm:
DFS

Select Heuristic:
H3

SOLVE

Result Info
Solution: RDLDIRDULDR
Cost: 360
Iterations: 26
Time: 35 ms
Step: 11/11

Controls:
[← →] Prev/Next
[ESC] Jump to step

< Prev Next >

	Iterasi	2	2	2	2	2	2	2	2	2
	Waktu (ms)	-	-	-	-	-	-	-	-	-
Test Case 6	Solusi	RDLU RULD R	RDLUR ULDR	RDLUR DULDR	RDLUR ULDR	RDLUR ULDR	RDLUR ULDR	RDLURU LDR	RDLUR ULDR	RDLDU RDULD R
	Cost	320	320	346	320	320	320	320	320	360
	Iterasi	31	19	18	30	30	30	30	27	26
	Waktu (ms)	62	66	35	50	64	33	44	43	88

Berdasarkan hasil pengujian pada beberapa test case, terlihat jelas perbedaan performa antara algoritma BFS, DFS, UCS, GBFS, dan A*. Algoritma *Depth-First Search* (DFS) dan *Breadth-First Search* (BFS) terbukti tidak selalu menghasilkan solusi yang optimal secara cost pergerakan. Hal ini terlihat sangat jelas pada Test Case 1, 2, dan 6, di mana DFS menghasilkan cost yang jauh lebih tinggi (berturut-turut 104, 425, dan 360) dibandingkan hasil dari algoritma optimal. Sifat algoritma ini memang dikhususkan untuk menemukan goal secepat mungkin tanpa mempertimbangkan *cost* yang membuatnya berhenti ketika solusi pertama ditemukan. DFS sendiri memiliki pola iterasi yang sangat bergantung pada keberuntungan dari struktur pohon pencarian (pohon status), terkadang iterasi yang dilakukan sangat sedikit karena kebetulan jalur yang ditelusuri langsung mengarah ke goal (seperti hanya 11 iterasi pada Test Case 1), namun di skenario lain iterasinya bisa sangat banyak jika terjebak dalam proses backtracking yang dalam.

Sebagai algoritma yang selalu menghasilkan solusi yang optimal, *Uniform Cost Search* (UCS) secara konsisten berhasil menemukan solusi dengan *cost* paling minimum pada semua *test case* yang memiliki penyelesaian (misalnya cost 87 pada Test Case 1 dan 337 pada Test Case 2). UCS bertumpu pada fungsi evaluasi $f(n) = g(n)$, di mana algoritma ini akan selalu mengekskansi *node* yang total biaya kumulatifnya yang paling minim dari titik awal pergerakan. Meskipun memberikan *cost* yang paling minimal, konsekuensi dari UCS adalah jumlah iterasinya yang cenderung lebih banyak dibandingkan dengan pendekatan heuristik, karena UCS harus memastikan seluruh kemungkinan langkah yang lebih murah telah dilewati sebelum beralih ke rute yang lebih mahal.

Di sisi lain, pemanfaatan heuristik pada algoritma Greedy Best-First Search (GBFS) menunjukkan performa penelusuran iterasi yang lebih cepat namun terkadang tidak mencapai optimalitas. Dengan fungsi evaluasi yang sepenuhnya bergantung pada perkiraan jarak ke tujuan atau $f(n) = h(n)$, GBFS cenderung mengambil langkah yang "terlihat" paling mendekati goal saat itu juga. Akibat sifat greedy ini, algoritma sering kali mengambil rute yang secara lokal tampak bagus namun secara global merugikan (*suboptimal*). Hal ini dibuktikan pada Test Case 2, di mana variasi heuristik H1 dan H3 dari GBFS menghasilkan cost 350 dan 425, lebih mahal dari

solusi optimal 337. Meski demikian, secara iterasi, algoritma ini memotong banyak cabang pencarian sehingga proses penelusurannya selesai dalam langkah yang lebih singkat.

Algoritma A* menggabungkan optimalitas UCS dan *heuristic* melalui fungsi $f(n) = g(n) + h(n)$. Hasil pengujian menunjukkan bahwa A* dengan heuristik 1 (H1) dan heuristik 2 (H2) selalu menghasilkan *cost* yang ekuivalen dengan UCS pada seluruh test case. Konsistensi ini membuktikan bahwa rumusan pendekatan H1 dan H2 bersifat *admissible* (tidak pernah melebihi-lebihkan atau *overestimate* biaya sebenarnya untuk mencapai tujuan), sehingga A* dijamin mencapai solusi paling optimal. Sementara itu, untuk rumusan Heuristik 3 (H3), meskipun pada test case ini kebetulan menghasilkan *cost* yang sama baiknya, secara teori belum tentu menjamin hasil yang optimal di setiap kondisi. Pada kasus dengan papan permainan yang berskala sangat luas dan mengandung banyak angka (0 hingga 9) yang harus dilewati satu per satu, pendekatan H3 memiliki probabilitas untuk melakukan *overestimate* perhitungan jarak. Namun, untuk papan dengan angka target yang sedikit, H3 masih tergolong *admissible*.

Terakhir, kasus-kasus yang tidak memiliki solusi, seperti yang terlihat pada Test Case 3, 4, dan 5, seluruh algoritma secara kompak langsung menghentikan penelusuran pada iterasi-iterasi paling awal (iterasi 1 atau 2). Hal ini membuktikan bahwa pohon status program berhasil mendeteksi atau mematikan *node node* yang tidak valid. Dengan demikian, program berhasil mengeksplorasi *node-node* hingga mencapai goal ataupun *game over* jika tidak ada cara untuk mencapai goal. Algoritma terbukti berhasil dalam mencari jalur dengan *cost* paling minimum.

LAMPIRAN

Seluruh source code dan berkas pendukung dalam pengerjaan tugas ini tersedia pada repositori GitHub yang dapat diakses melalui tautan berikut:

https://github.com/Jerannn24/Tucil3_13524005_13524055

Pernyataan tidak melakukan kecurangan yang ditandatangani dengan format sebagai berikut:

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Geraldo Artemius- 13524005



Junior Natra Situmorang - 13524055

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain	✓	

	yang utama (Heuristik X dan Heuristik Y)		
--	--	--	--